

+

# Informatik 2 – Verkettete Listen & Bäume

K

A



©image: various

## Heute

- Dialogsysteme
- Verhaltensbäume
- Entscheidungsbäume
- Binärer Suchbaum
- Listen

## C++

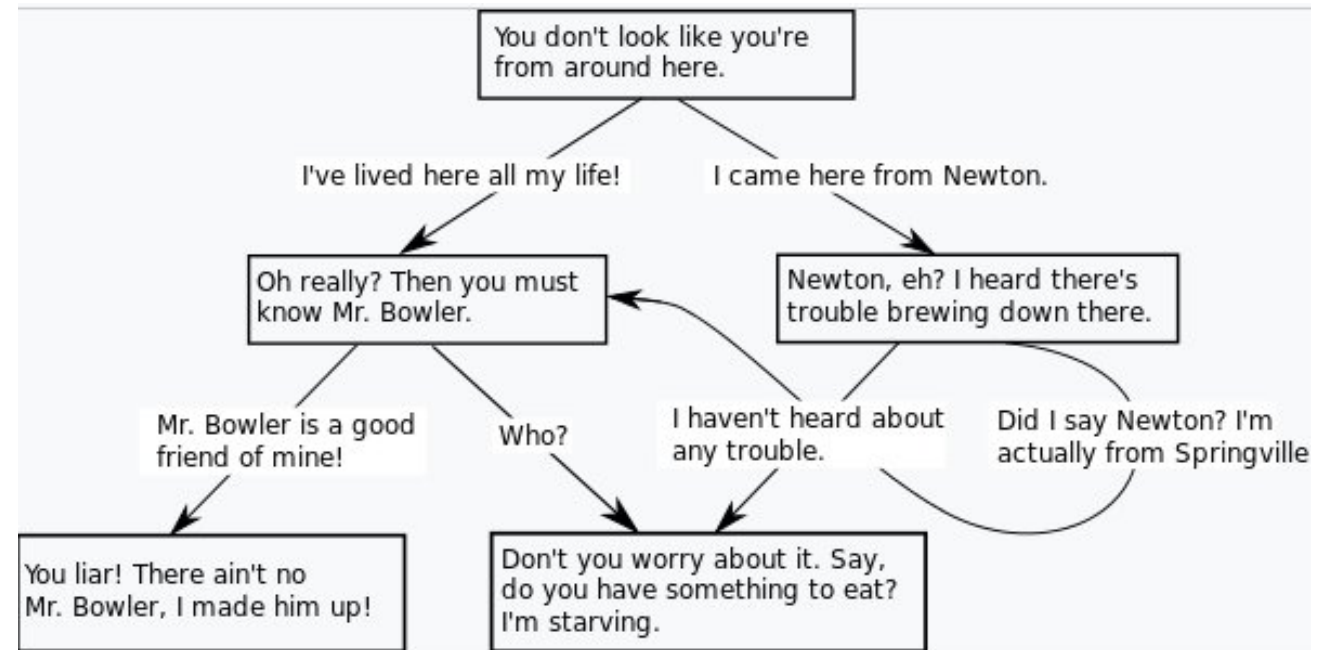
- struct
- Verkettete Listen
- Bäume
- for each



<https://youtu.be/azheyIJrvvk>

# Dialogue Tree

- A dialogue tree, or conversation tree, is a gameplay mechanic that is used throughout many adventure/action/role-play games
- interacting with a NPC

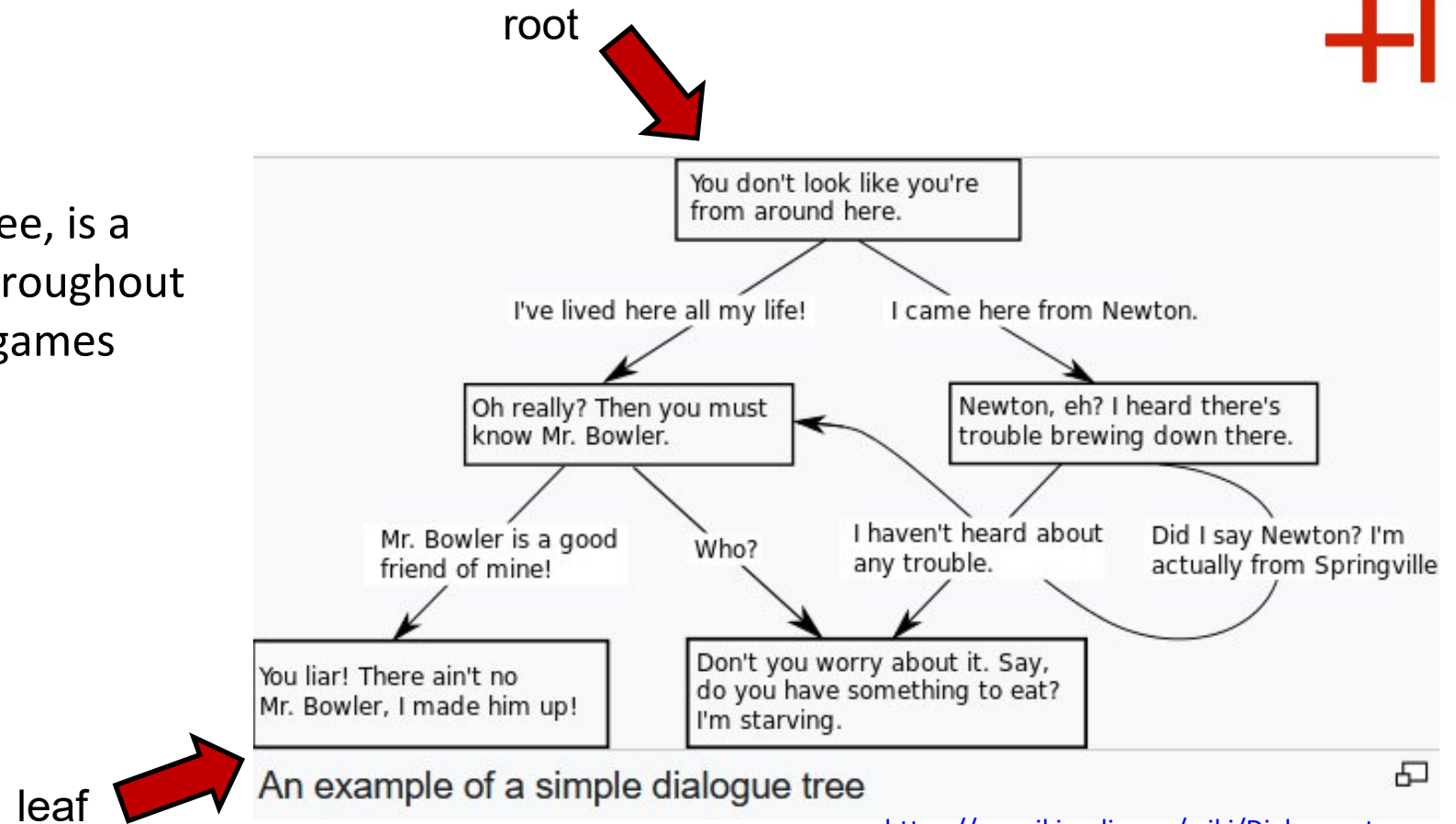


An example of a simple dialogue tree

[https://en.wikipedia.org/wiki/Dialogue\\_tree](https://en.wikipedia.org/wiki/Dialogue_tree)

# Dialogue Tree

- A dialogue tree, or conversation tree, is a gameplay mechanic that is used throughout many adventure/action/role-play games
- interacting with a NPC



# Dialogue Tree – Unreal Engine 5



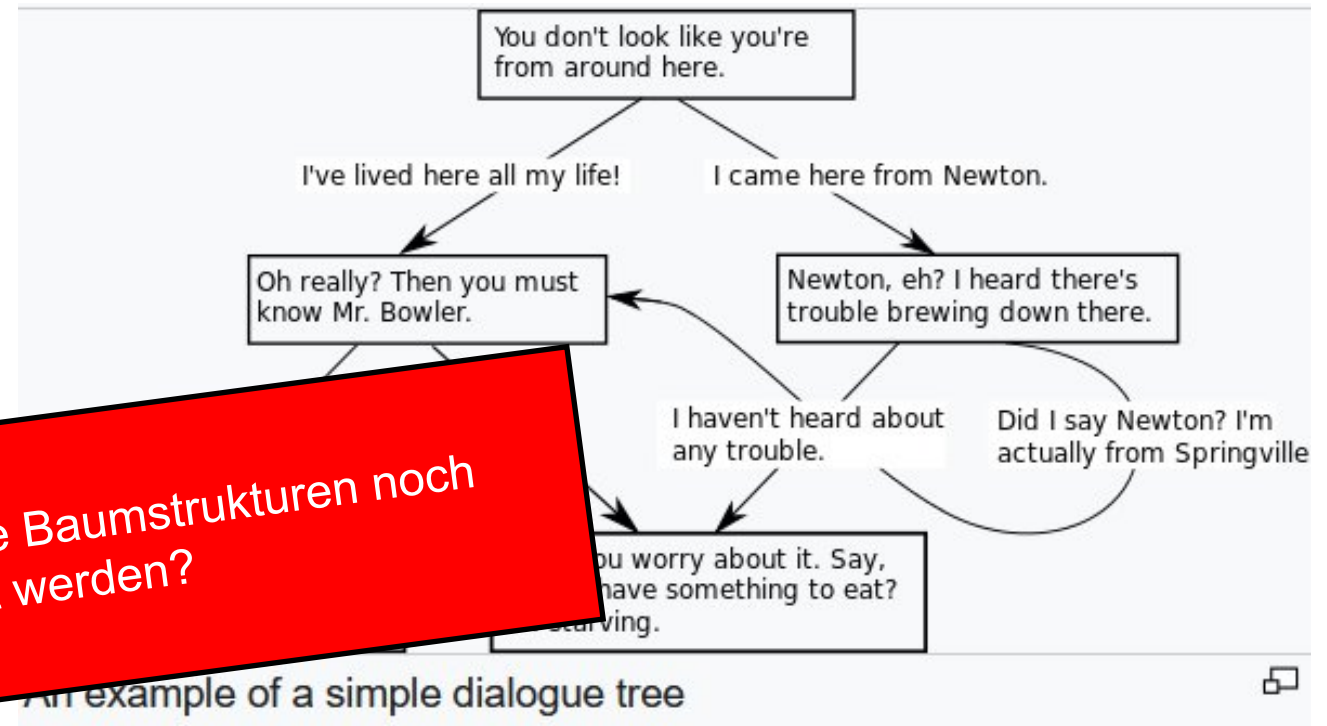
<https://youtu.be/azheyIJrvvk>



# Dialogue Tree

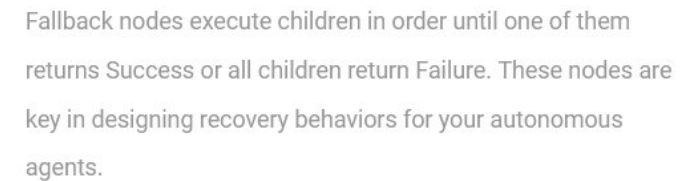
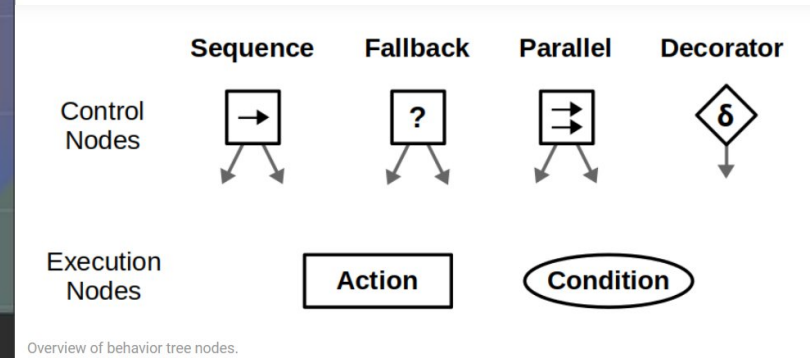


- A dialogue tree, or conversation tree, is a gameplay mechanic that is used throughout many adventure/action/role-play games
- interacting with a NPC



Wo/wofür können solche Baumstrukturen noch eingesetzt werden?

[https://en.wikipedia.org/wiki/Dialogue\\_tree](https://en.wikipedia.org/wiki/Dialogue_tree)



[2] Sebastian Castro (2021), Introduction to behavior trees <https://robohub.org/introduction-to-behavior-trees/>

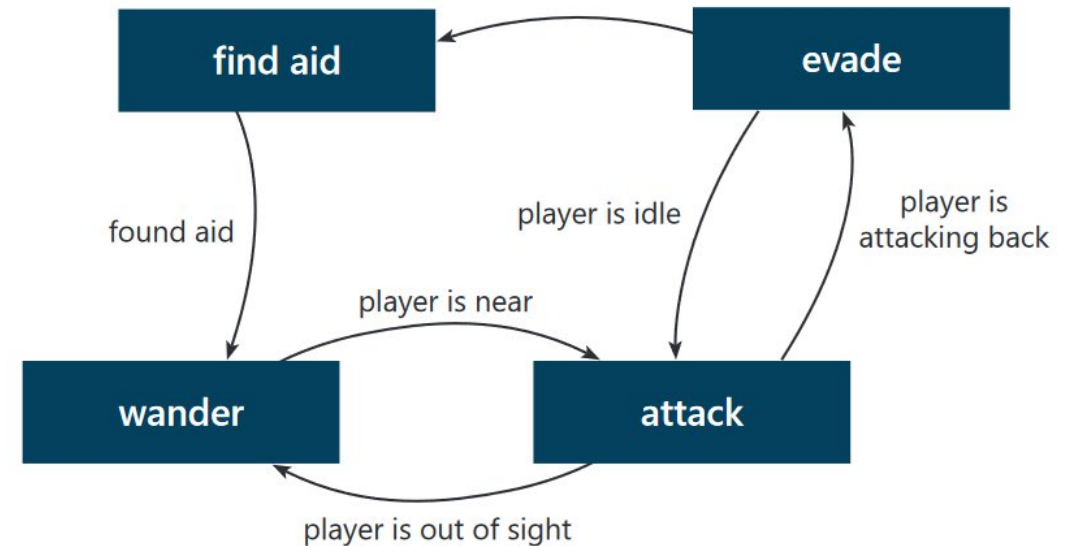


# Verhaltensbäume



Verhaltensmodelle (finite-state models), wie z.B. die

- Pac-Man Ghosts
- Gegner bei Wolfenstein, Doom, etc.



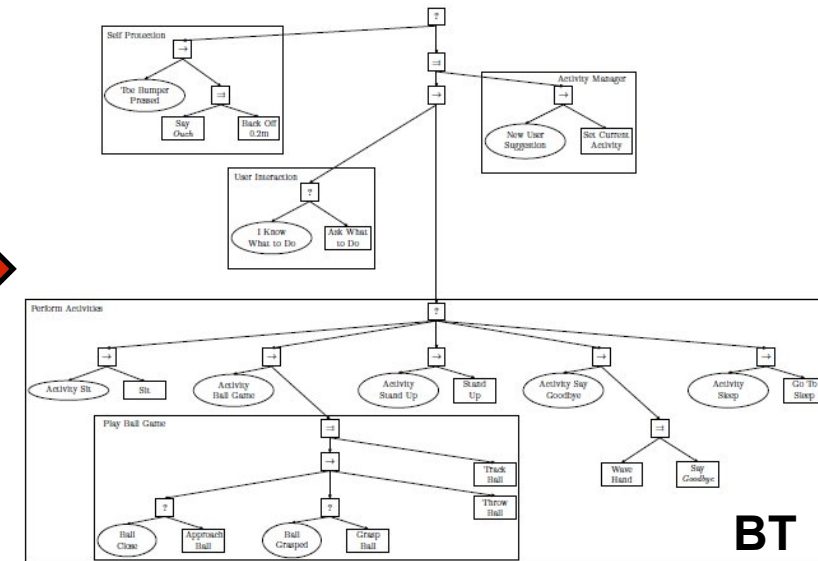
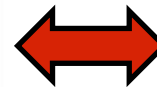
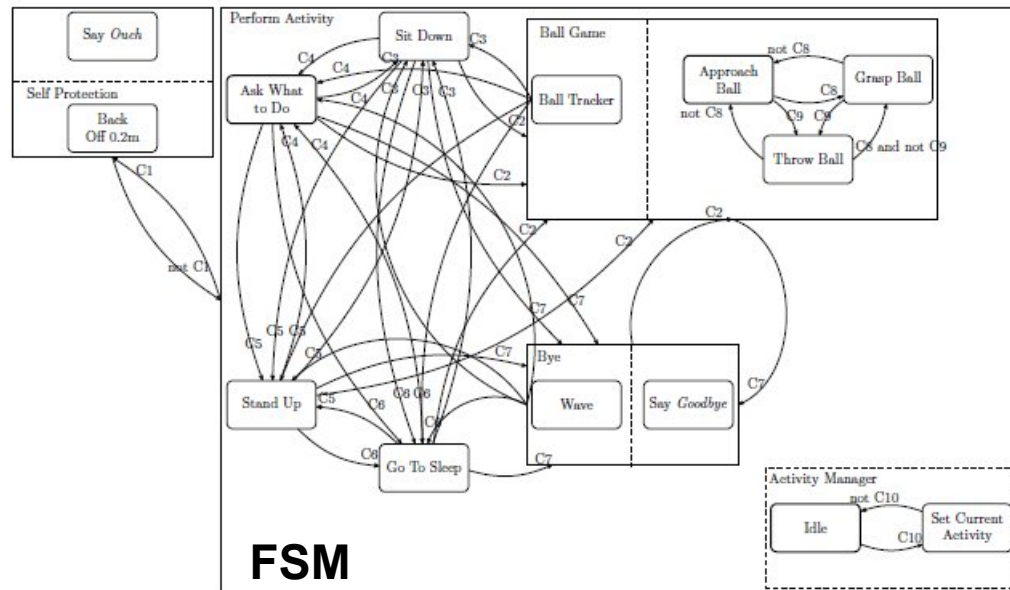
# Verhaltensbäume



Verhaltensmodelle (finite-state models), wie z.B. die

- Pac-Man Ghosts
- Gegner bei Wolfenstein, Doom, etc.

können mit Verhaltensbäumen abstrahiert werden.



- [1] Game Lab at Newcastle University: Artificial Intelligence 1: Finite State Machines  
<https://research.ncl.ac.uk/game/mastersdegree/gametechnologies/previousinformation/artificialintelligence1finitestatemachines/2016%20Tutorial%208%20-%20Finite%20State%20Machines.pdf>
- [2] Laura E Shummon Maass Artificial Intelligence in Video Games <https://towardsdatascience.com/artificial-intelligence-in-video-games-3e2566d59c22>
- [3] Colledanchise and Ogren (2022), Behavior Trees in Robotics and AI, arXiv:1709.00084v6 [cs.RO] <https://arxiv.org/pdf/1709.00084.pdf>
- [4] Sebastian Castro (2021), Introduction to behavior trees <https://robohub.org/introduction-to-behavior-trees/>

# BTW: Wieso ist die KI in Spielen meist so dumm?



*“We could have made the [AI] enemies much smarter [..]  
but it’s not as a good of an idea as a lot of people might”*

*- John Carmack, Doom 30<sup>th</sup> anniversary stream*

*“More of something dumb is preferable to super smart AI”*

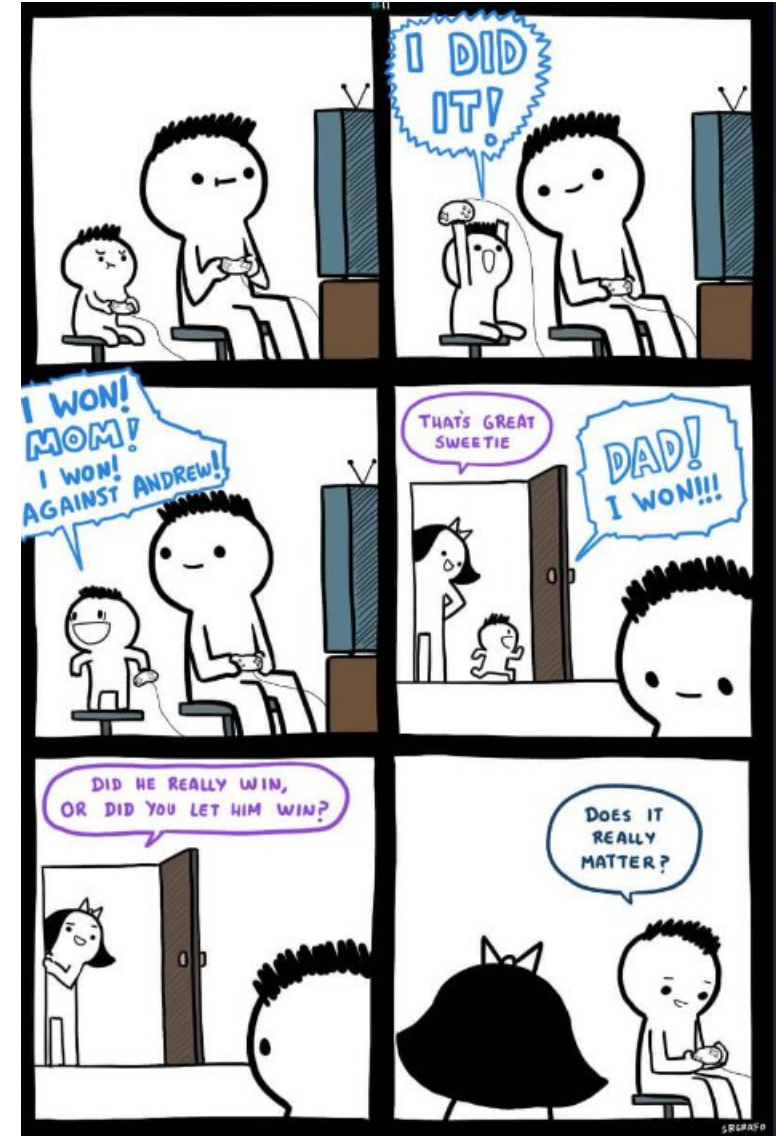
*- John Romero, Doom 30<sup>th</sup> anniversary stream*

<https://www.twitch.tv/videos/2000693432>

# BTW: Wieso ist die KI in Spielen meist so dumm?



Ein perfekter Gegner macht keinen Spaß....



# BTW: Wieso ist die KI in Spielen meist so dumm?

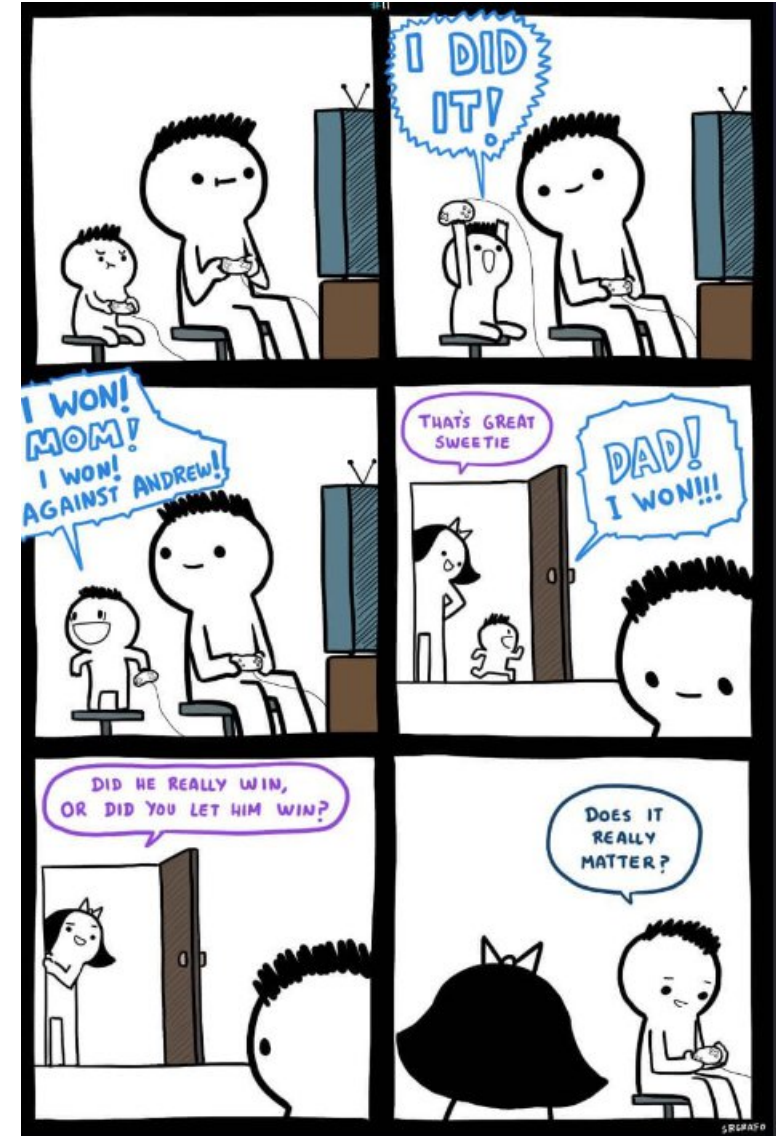


Ein perfekter Gegner macht keinen Spaß....

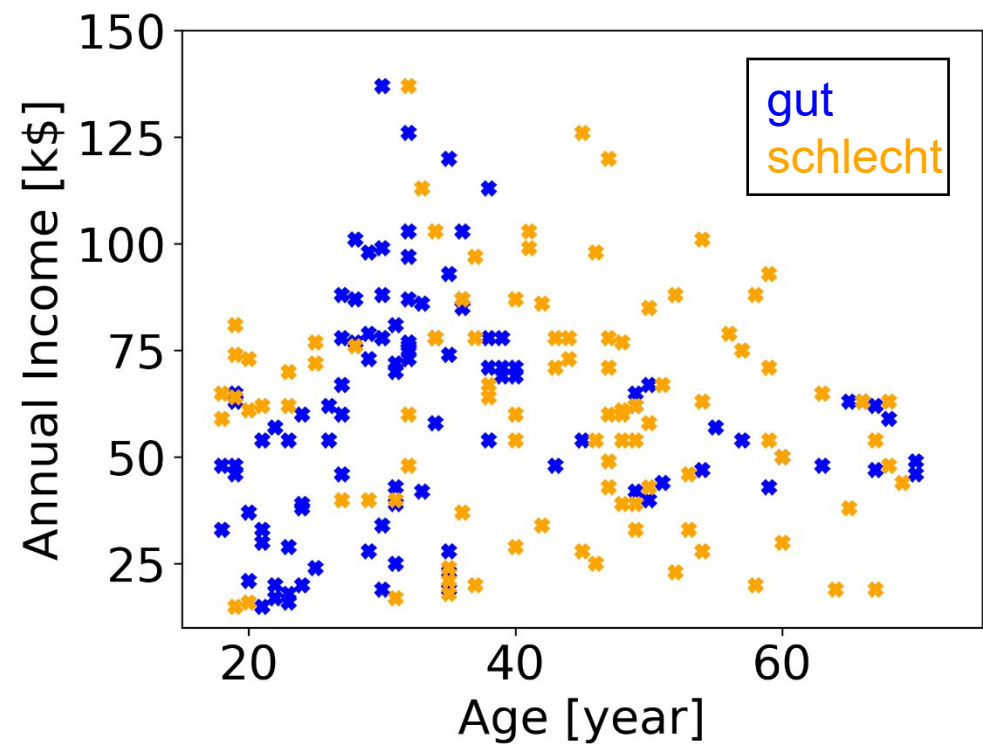
- The bigger rat needs to let the smaller rat win at least 30 percent of the time.
- It's the smaller rat that asks the bigger rat for a fight or a match.
- If it doesn't win at least 3 out of 10 matches, it will stop asking for matches. So the fun is over for both of them. [1]

[1] William Peynsaert (2017) citing Jordan Peterson – lecture

<https://projectauthenticity.org/2017/11/06/the-surprising-moral-dimension-of-fights-between-rats-strong-rats-have-to-let-their-weaker-opponent-win-30-percent-of-the-time-if-they-dont/>



# Entscheidungsbäume

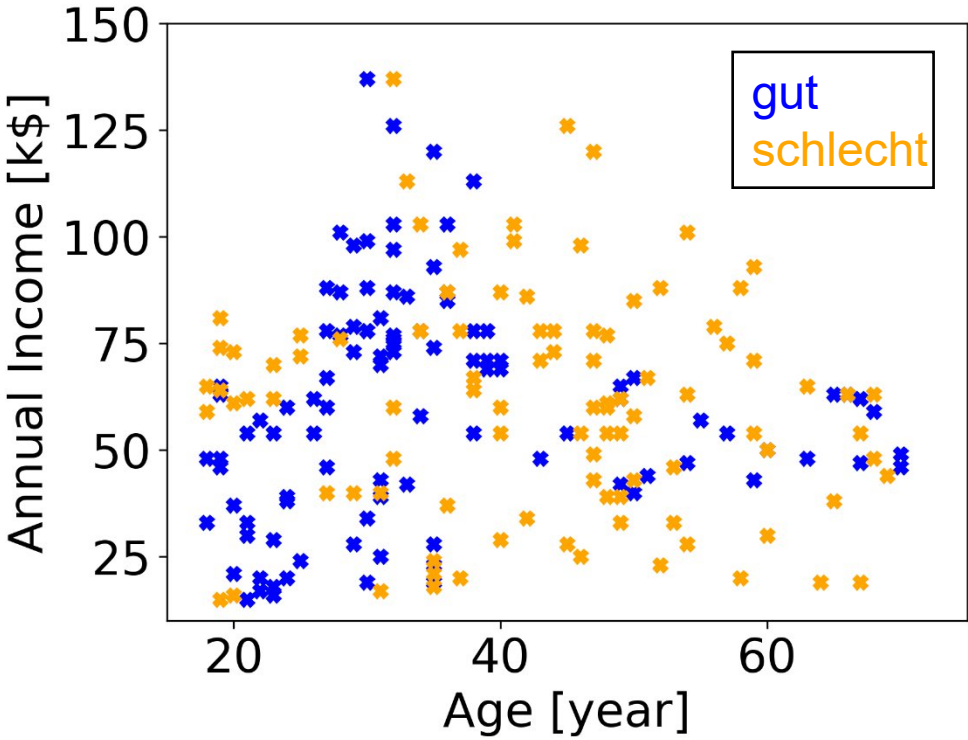


| CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|------------|--------|-----|---------------------|------------------------|
| 1          | Male   | 19  | 15                  | schlecht               |
| 2          | Male   | 21  | 15                  | gut                    |
| 3          | Female | 20  | 16                  | schlecht               |
| 4          | Female | 23  | 16                  | gut                    |
| 5          | Female | 31  | 17                  | schlecht               |



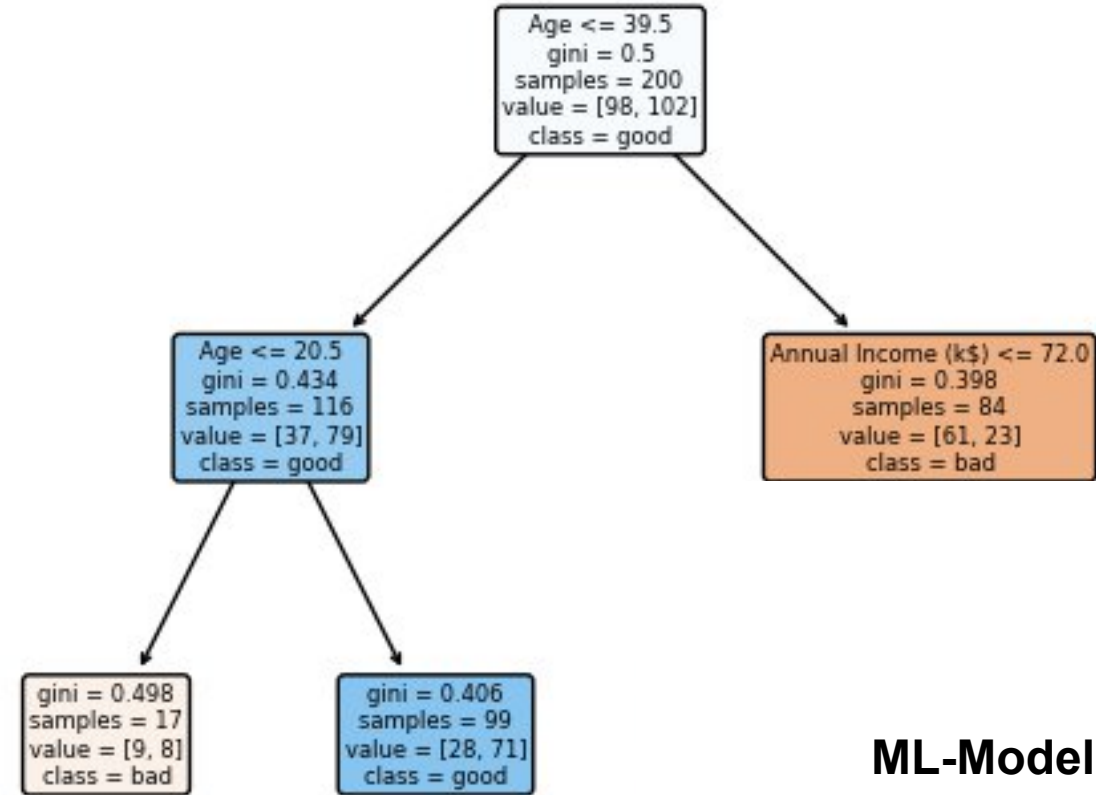
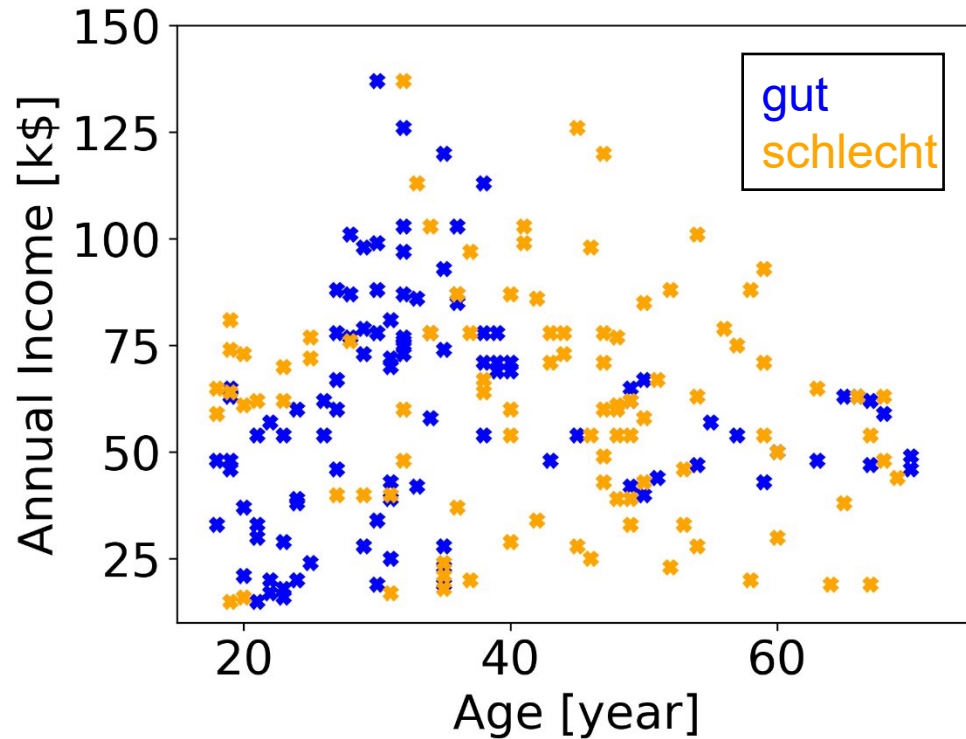


Wie können wir vorhersagen, welcher neue Kunde viel Geld ausgeben wird?



| CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|------------|--------|-----|---------------------|------------------------|
| 1          | Male   | 19  | 15                  | schlecht               |
| 2          | Male   | 21  | 15                  | gut                    |
| 3          | Female | 20  | 16                  | schlecht               |
| 4          | Female | 23  | 16                  | gut                    |
| 5          | Female | 31  | 17                  | schlecht               |

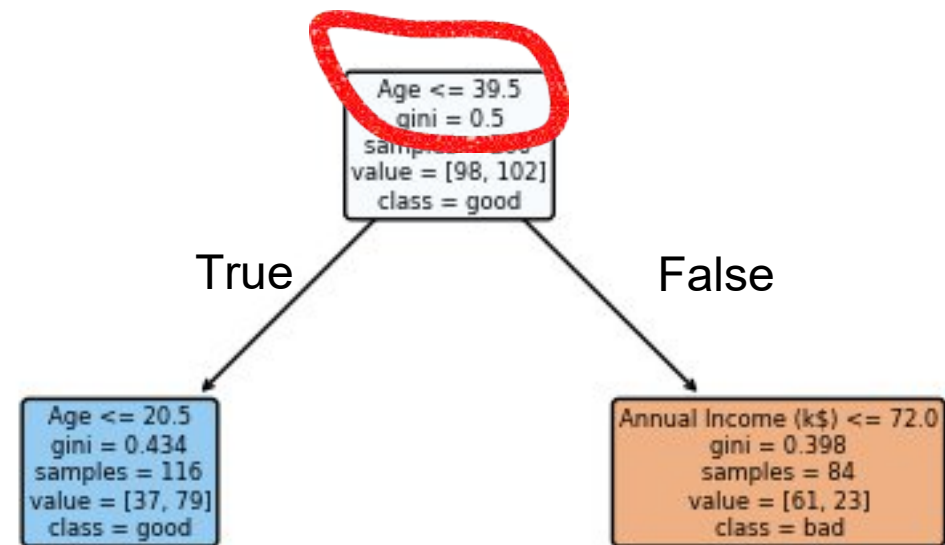
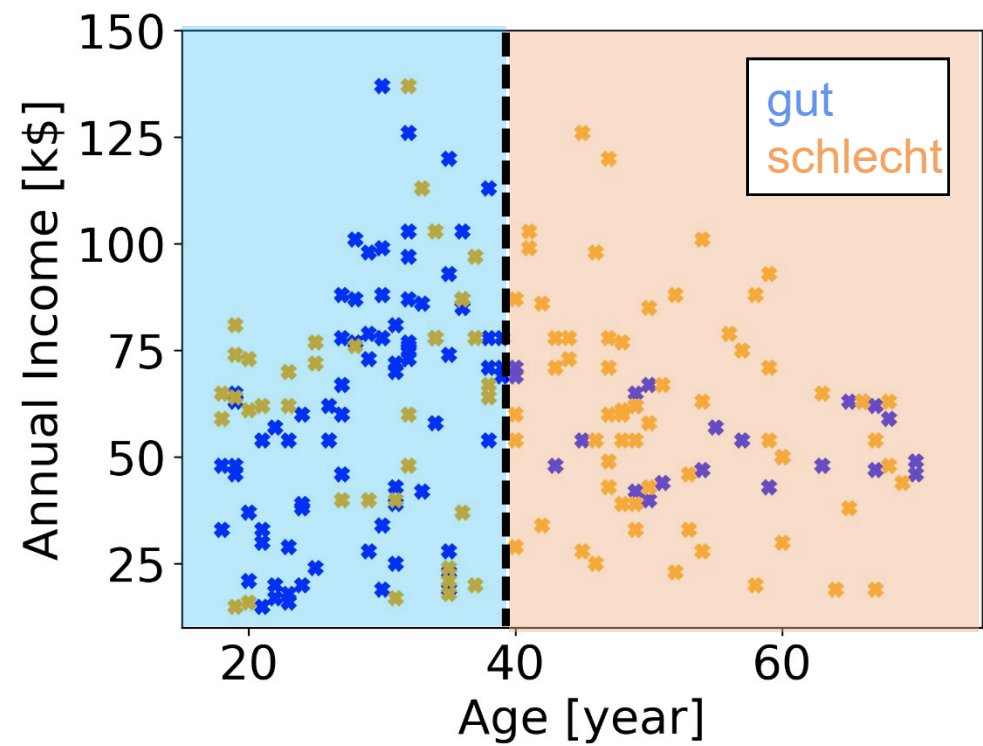
# Entscheidungsbäume



ML-Modell

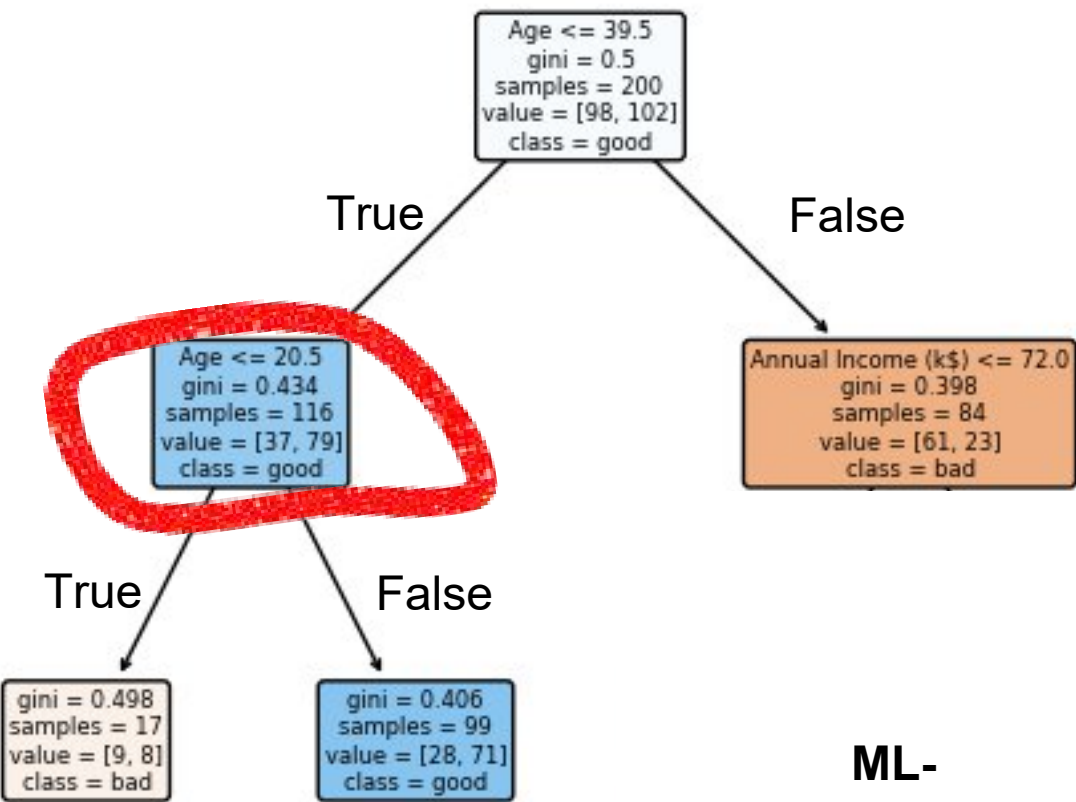
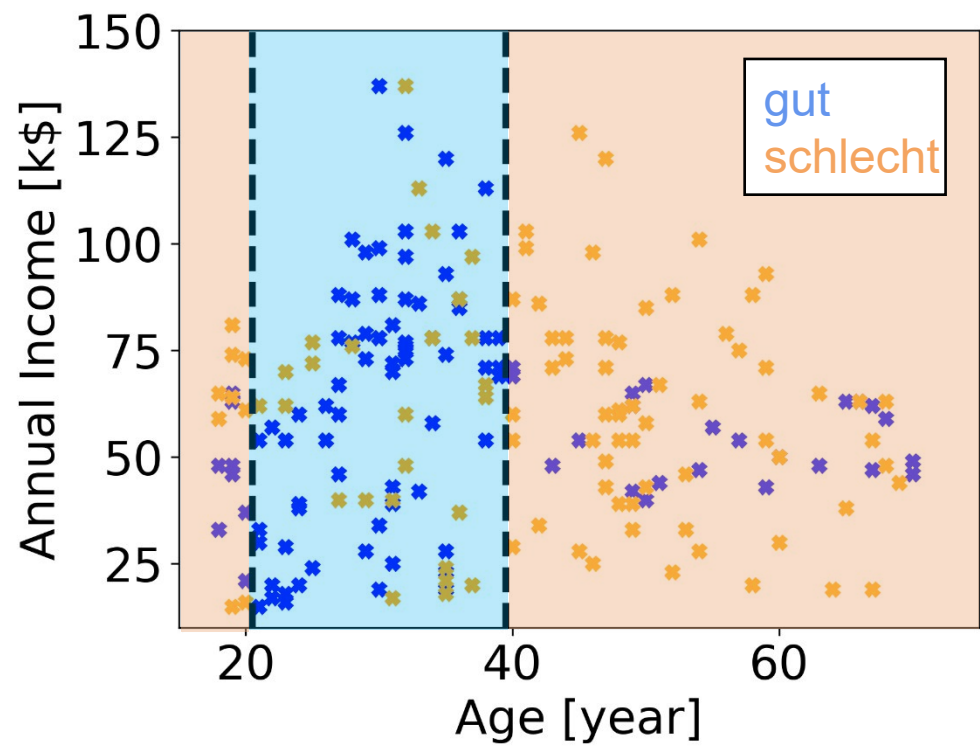
```
from sklearn import tree
clf = tree.DecisionTreeClassifier(max_depth=2)
clf = clf.fit(X_OneHot, y_label)
```

# Entscheidungsbäume



ML-  
Modell

# Entscheidungsbäume



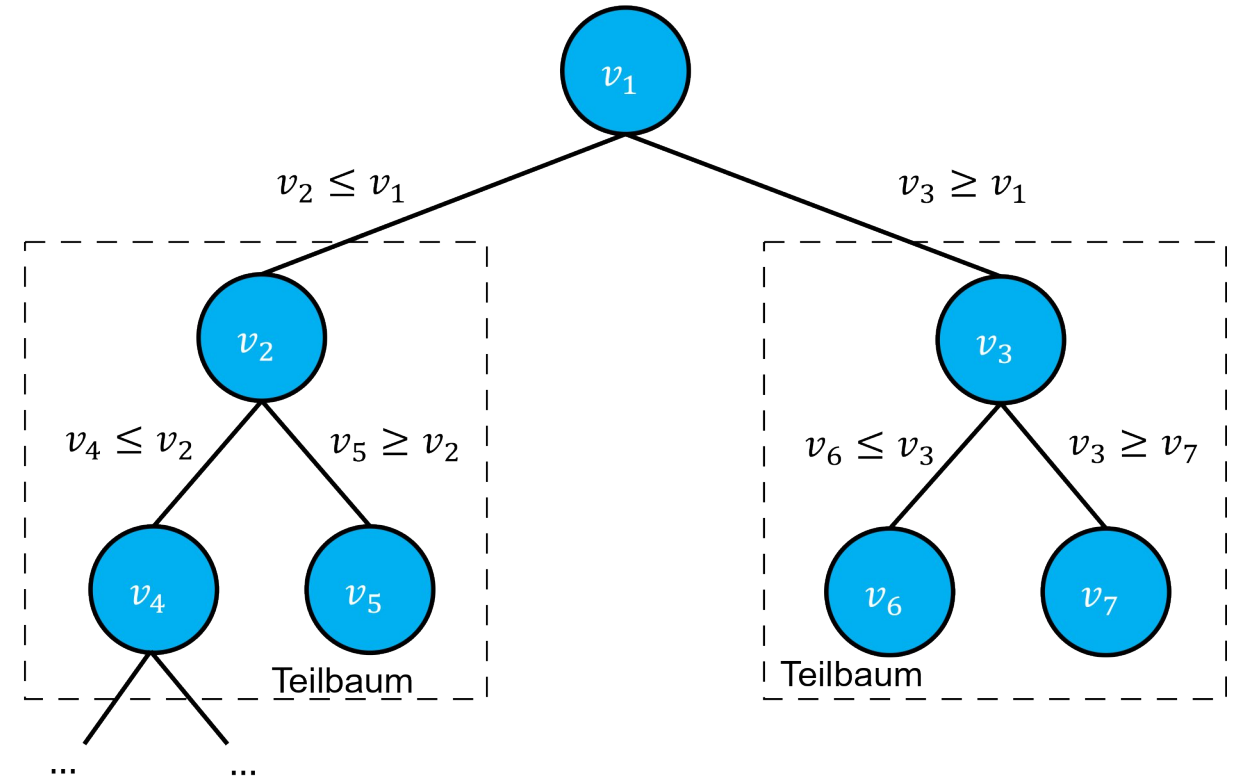
ML-  
Modell

Geordnete Baumstruktur:

Ein binärer Suchbaum ist ein Baum, in welchem jeder Knoten (Schlüssel) zwei Nachfolger hat und für welchen gilt, dass der Wurzelschlüssel  $v_w$  stets größer gleich seines linken Nachfolgerknotens  $v_l$  ist und  $v_w$  stets kleiner gleich seines rechten Nachfolgerknotens  $v_r$  ist.

Es gilt also:

$$v_l \leq v_w \wedge v_r \geq v_w$$



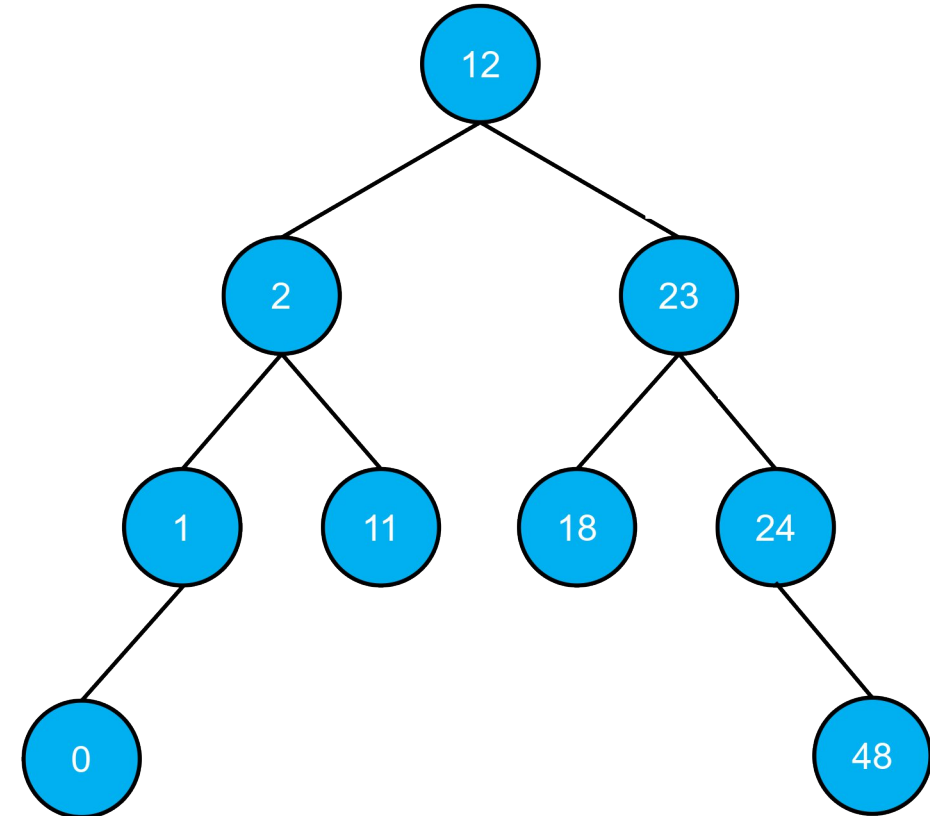
$l = \langle 12, 2, 23, 1, 11, 18, 24, 0, 48 \rangle$

Geordnete Baumstruktur:

Ein binärer Suchbaum ist ein Baum, in welchem jeder Knoten (Schlüssel) zwei Nachfolger hat und für welchen gilt, dass der Wurzelschlüssel  $v_w$  stets größer gleich seines linken Nachfolgerknotens  $v_l$  ist und  $v_w$  stets kleiner gleich seines rechten Nachfolgerknotens  $v_r$  ist.

Es gilt also:

$$v_l \leq v_w \wedge v_r \geq v_w$$





# Binärer Suchbaum



Suche **sehr schnell** möglich.  
Z.B. ist die Zahl 24 enthalten?

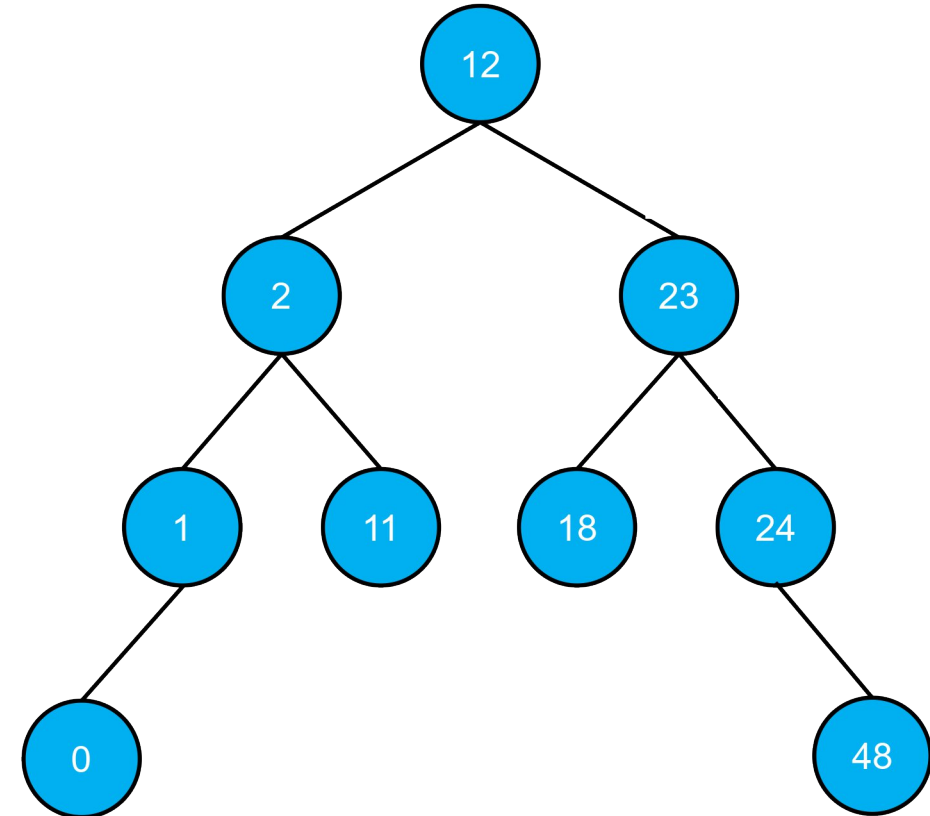
$l = \langle 12, 2, 23, 1, 11, 18, 24, 0, 48 \rangle$

Geordnete Baumstruktur:

Ein binärer Suchbaum ist ein Baum, in welchem jeder Knoten (Schlüssel) zwei Nachfolger hat und für welchen gilt, dass der Wurzelschlüssel  $v_w$  stets größer gleich seines linken Nachfolgerknotens  $v_l$  ist und  $v_w$  stets kleiner gleich seines rechten Nachfolgerknotens  $v_r$  ist.

Es gilt also:

$$v_l \leq v_w \wedge v_r \geq v_w$$



# Binärer Suchbaum



Suche **sehr schnell** möglich.  
Z.B. ist die Zahl 24 enthalten?

Geordnete Baumstruktur:

Ein binärer Suchbaum ist ein Baum, in welchem jeder Knoten (Schlüssel) zwei Nachfolger hat und für welchen gilt, dass der Wurzelschlüssel  $v_w$  stets größer gleich seines linken Nachfolgerknotens  $v_l$  ist und  $v_w$  stets kleiner gleich seines rechten Nachfolgerknotens  $v_r$  ist.

Es gilt also:

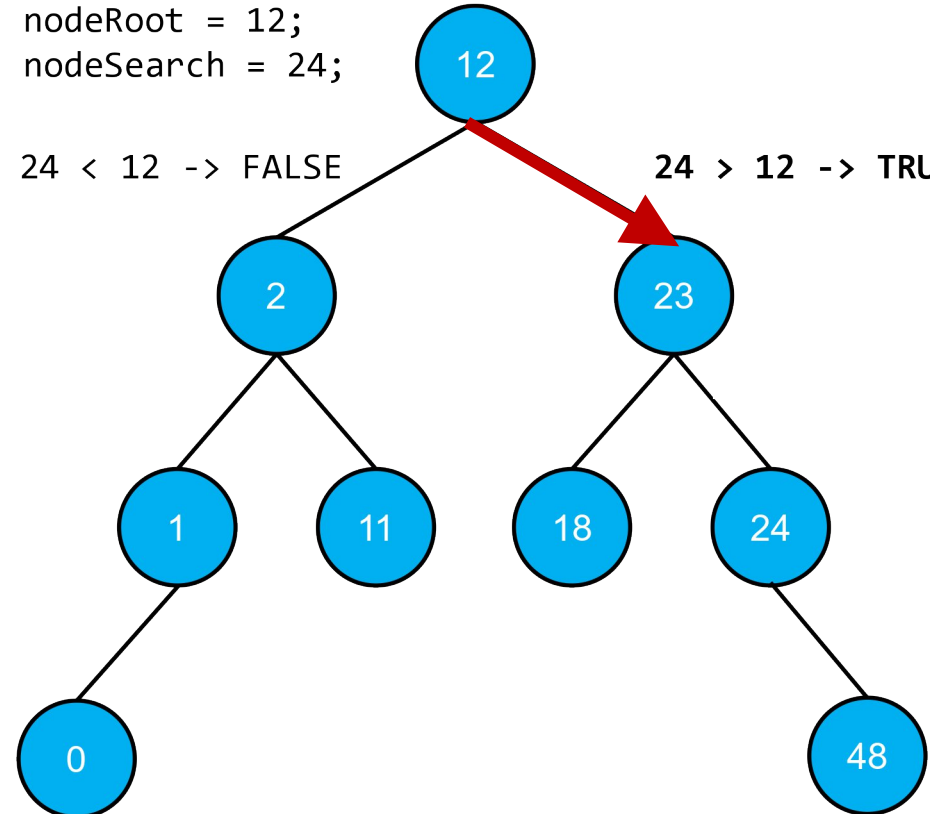
$$v_l \leq v_w \wedge v_r \geq v_w$$

$l = \langle 12, 2, 23, 1, 11, 18, 24, 0, 48 \rangle$

nodeRoot = 12;  
nodeSearch = 24;

$24 < 12 \rightarrow \text{FALSE}$

$24 > 12 \rightarrow \text{TRUE}$



# Binärer Suchbaum



Suche **sehr schnell** möglich.  
Z.B. ist die Zahl 24 enthalten?

Geordnete Baumstruktur:

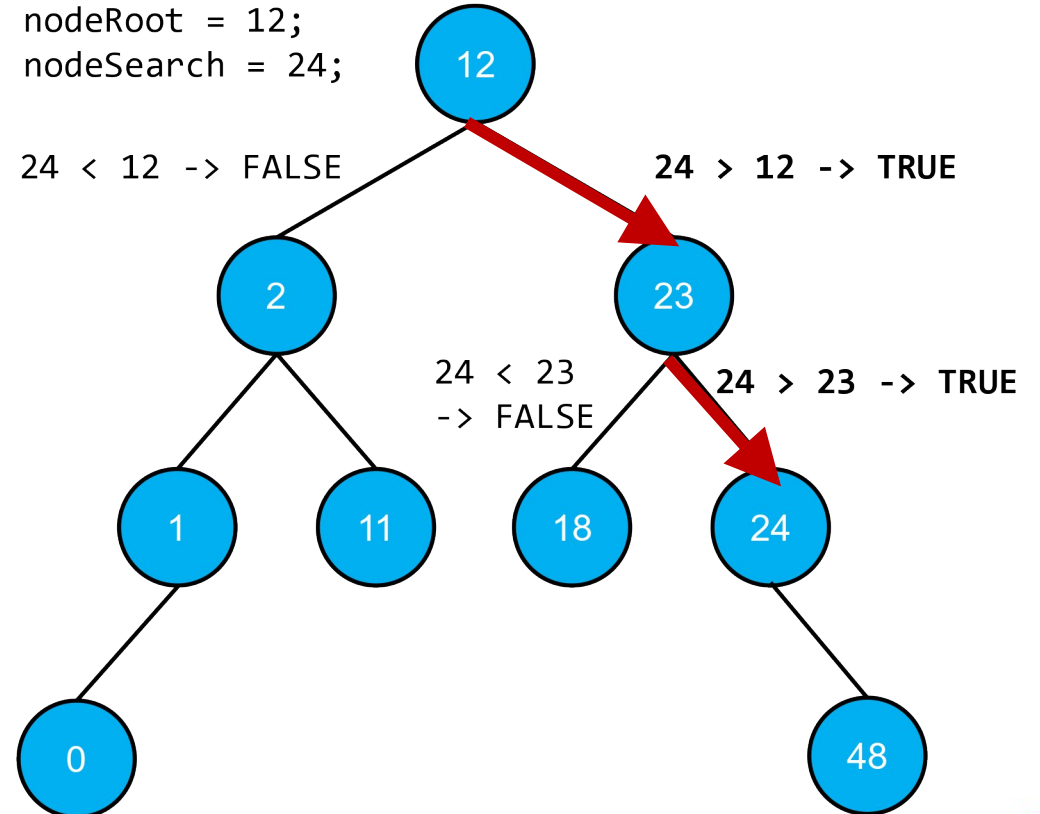
Ein binärer Suchbaum ist ein Baum, in welchem jeder Knoten (Schlüssel) zwei Nachfolger hat und für welchen gilt, dass der Wurzelschlüssel  $v_w$  stets größer gleich seines linken Nachfolgerknotens  $v_l$  ist und  $v_w$  stets kleiner gleich seines rechten Nachfolgerknotens  $v_r$  ist.

Es gilt also:

$$v_l \leq v_w \wedge v_r \geq v_w$$

$l = \langle 12, 2, 23, 1, 11, 18, 24, 0, 48 \rangle$

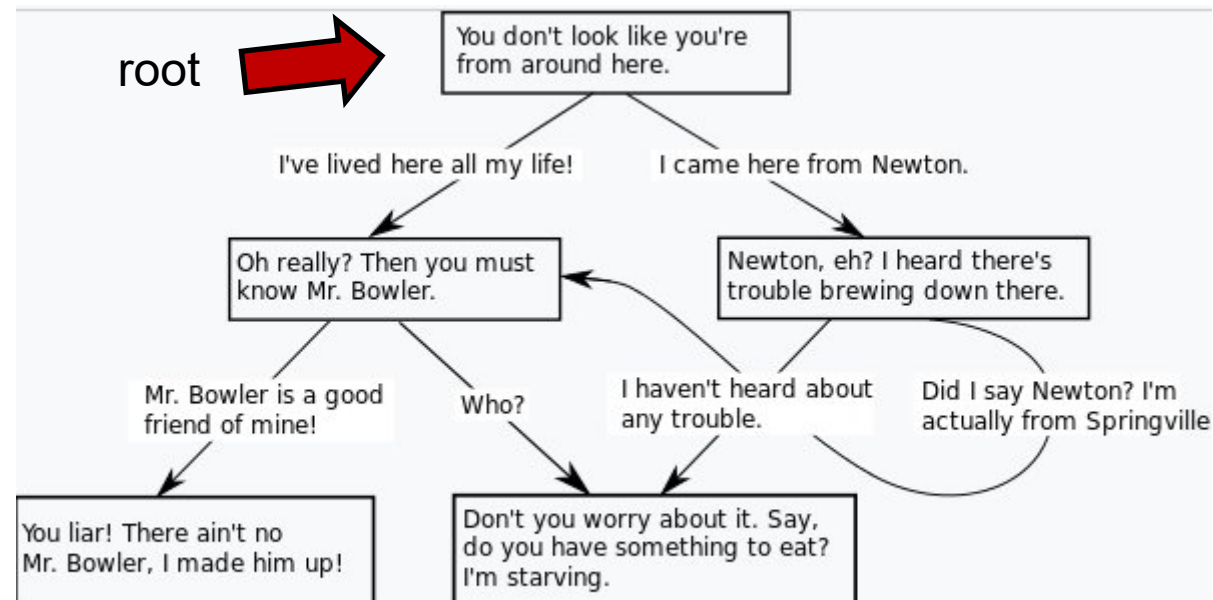
nodeRoot = 12;  
nodeSearch = 24;



# Baumstruktur - Grundbegriffe



- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine **Wurzel (root)**  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind

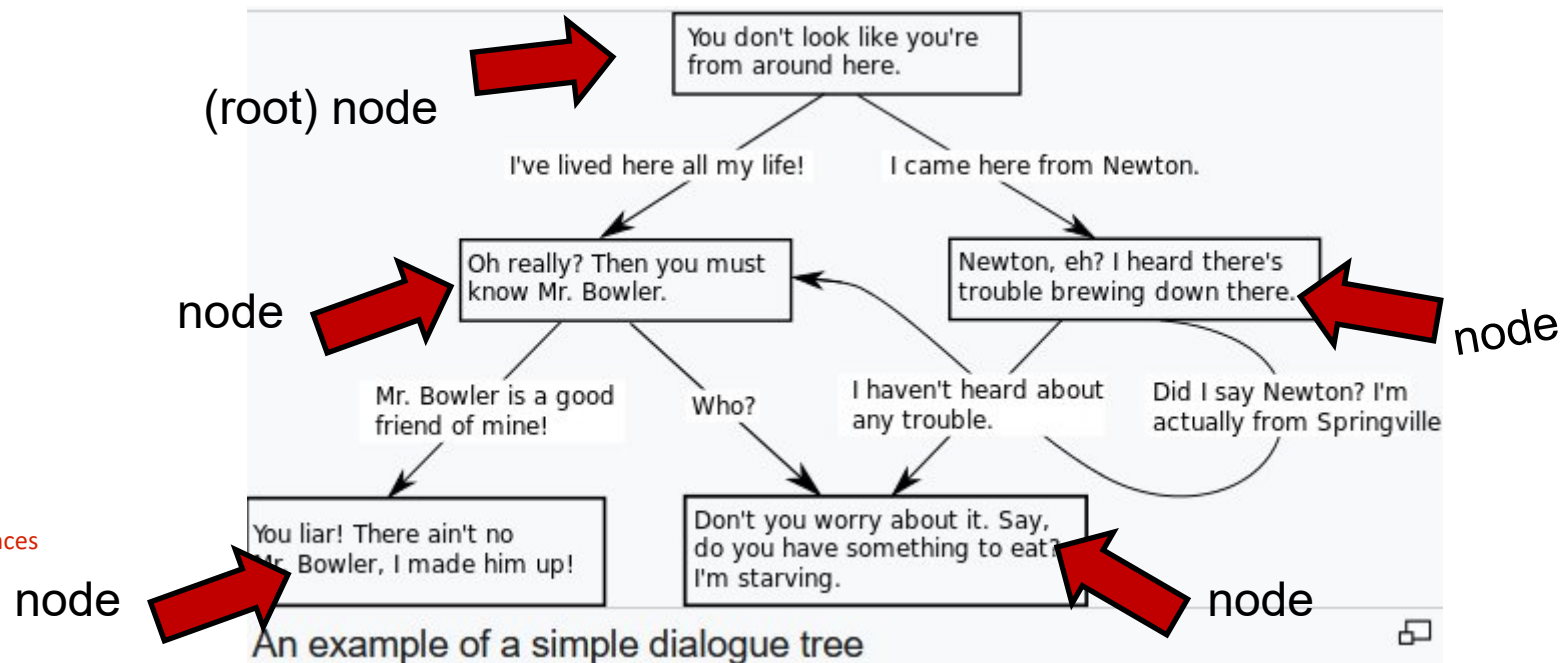


An example of a simple dialogue tree

# Baumstruktur - Grundbegriffe



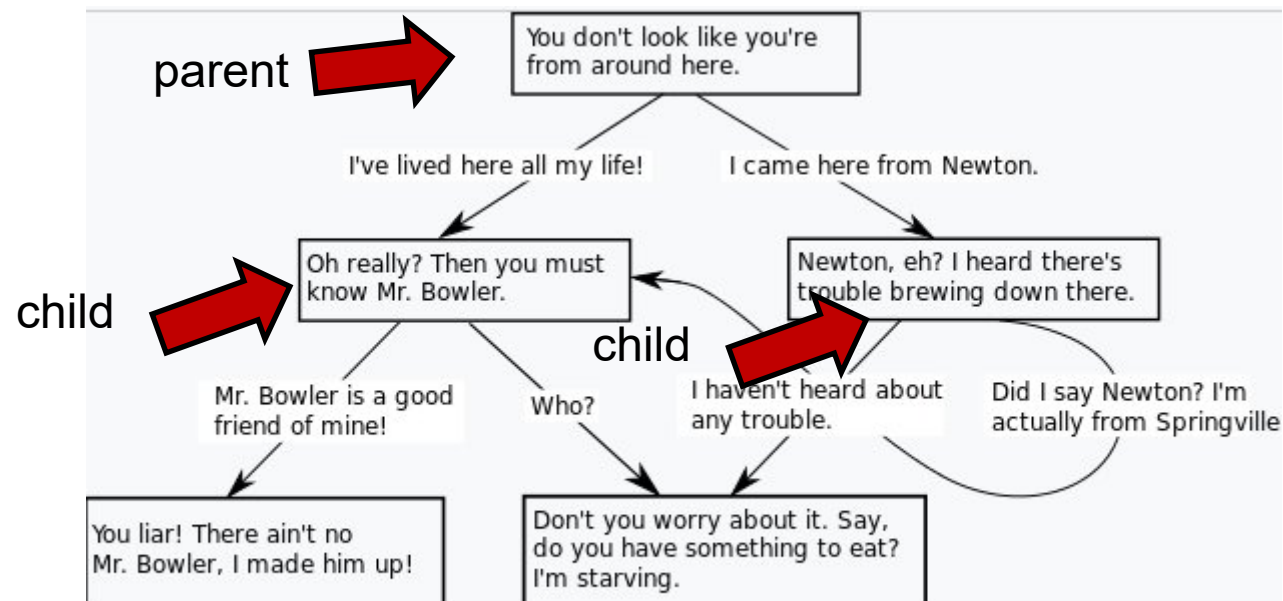
- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine **Wurzel (root)**  $w$  von welcher aus alle **Knoten (nodes)**  $V$  erreichbar sind



# Baumstruktur - Grundbegriffe



- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine Wurzel (root)  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind
  - Jeder Knoten (**Kind**) (außer  $w$ ) hat genau einen Vorgänger (**Vater**), der eine Ebene über diesem ist



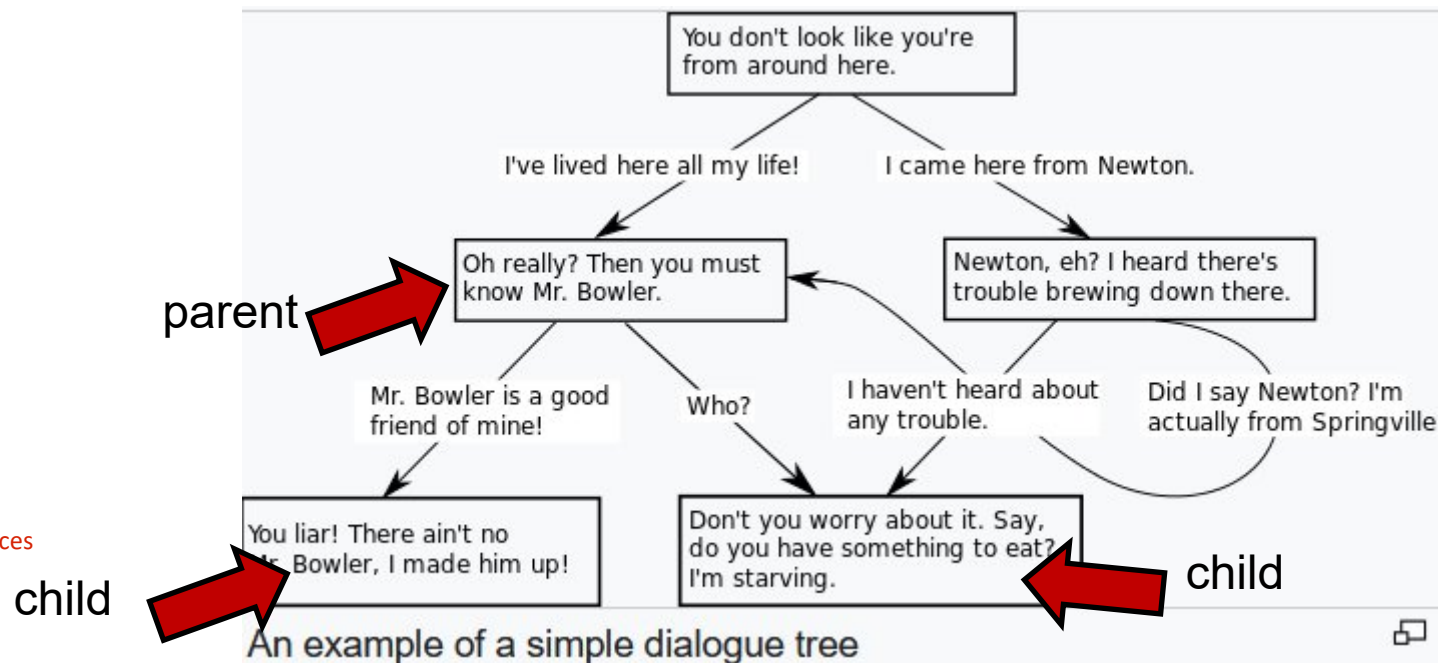
An example of a simple dialogue tree



# Baumstruktur - Grundbegriffe



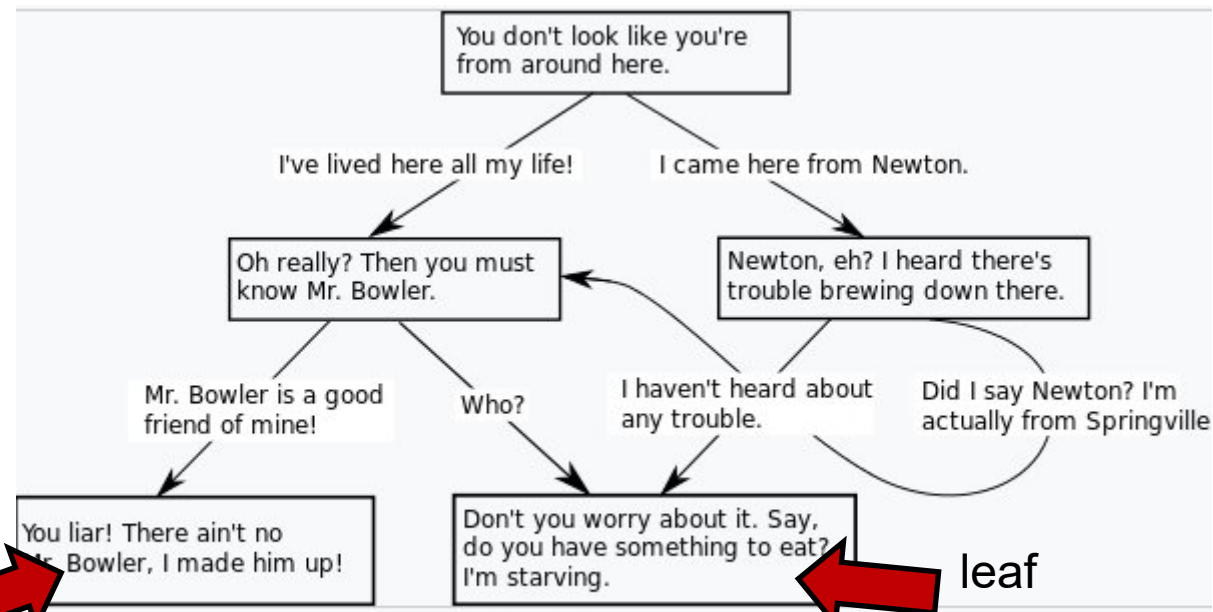
- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine Wurzel (root)  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind
  - Jeder Knoten (**Kind**) (außer  $w$ ) hat genau einen Vorgänger (**Vater**), der eine Ebene über diesem ist



# Baumstruktur - Grundbegriffe



- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine Wurzel (root)  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind
  - Jeder Knoten (Kind) (außer  $w$ ) hat genau einen Vorgänger (Vater), der eine Ebene über diesem ist
  - Die Höhe  $h$  eines Baumes ergibt sich durch die höchste Ebene  $E$  eines **Blattes** mit  $h = E + 1$
  - Die maximale Anzahl an Kanten eines Baumes ist  $\|E\|_{max} = n - 1$  für  $n = \|V\|$

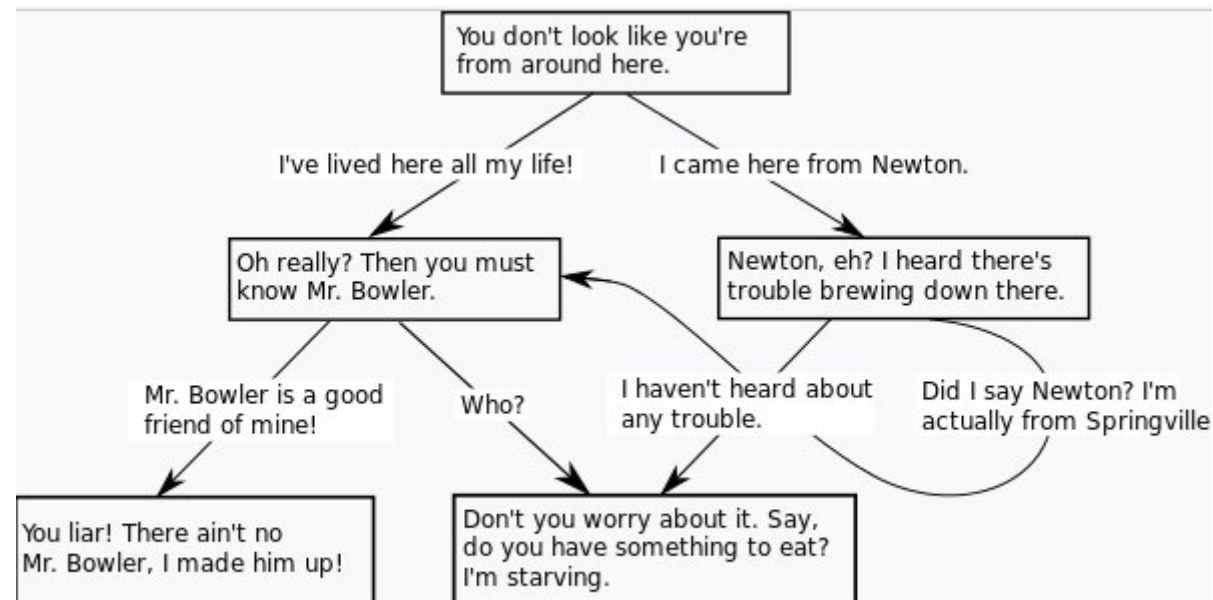


# Baumstruktur - Grundbegriffe



- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten  $v, w \in V$  sind durch die Verknüpfung  $(v, w) \in E$  verbunden. Ein Baum ist ein zusammenhängender Graph, in dem zwischen je zwei Knoten  $v, w \in V$  genau ein Pfad existiert, der  $v$  mit  $w$  verbindet. Jeder Knoten  $v \in V$  ist von einem Knoten  $u \in V$  erreichbar, den die Wurzel des Baumes nennt man. Die Höhe  $h$  eines Baumes ergibt sich durch die höchste Ebene  $E$  eines Blattes mit  $h = E + 1$ . Die maximale Anzahl an Kanten eines Baumes ist  $\|E\|_{max} = n - 1$  für  $n = \|V\|$ .

Wieso ist der dialogue tree unten gar kein echter Baum?



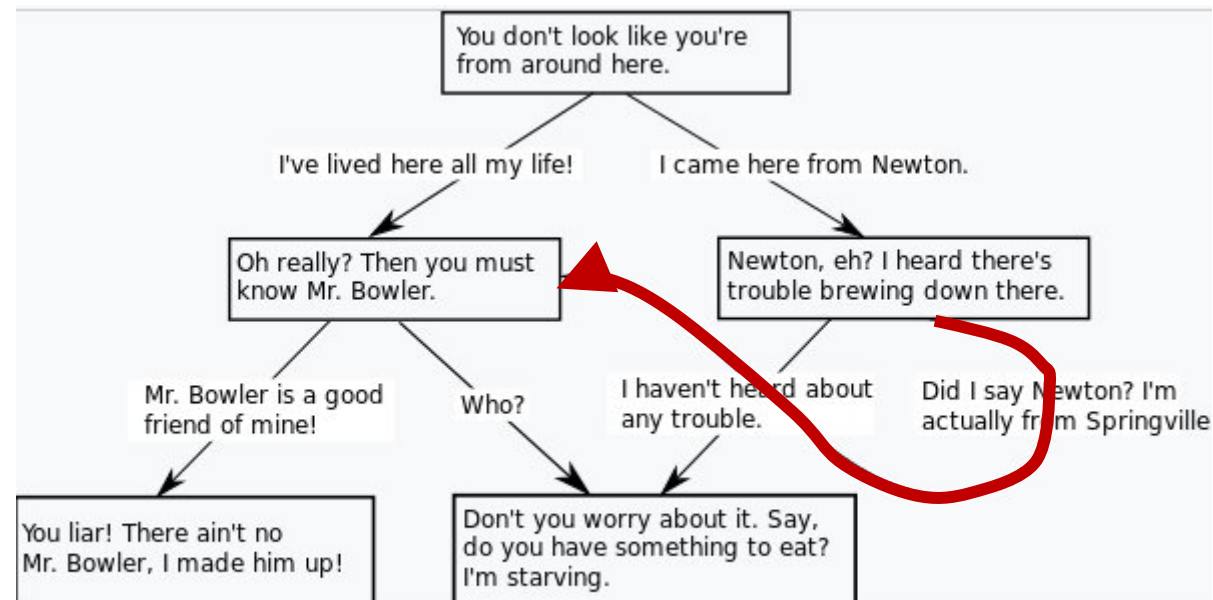
An example of a simple dialogue tree

# Baumstruktur - Grundbegriffe



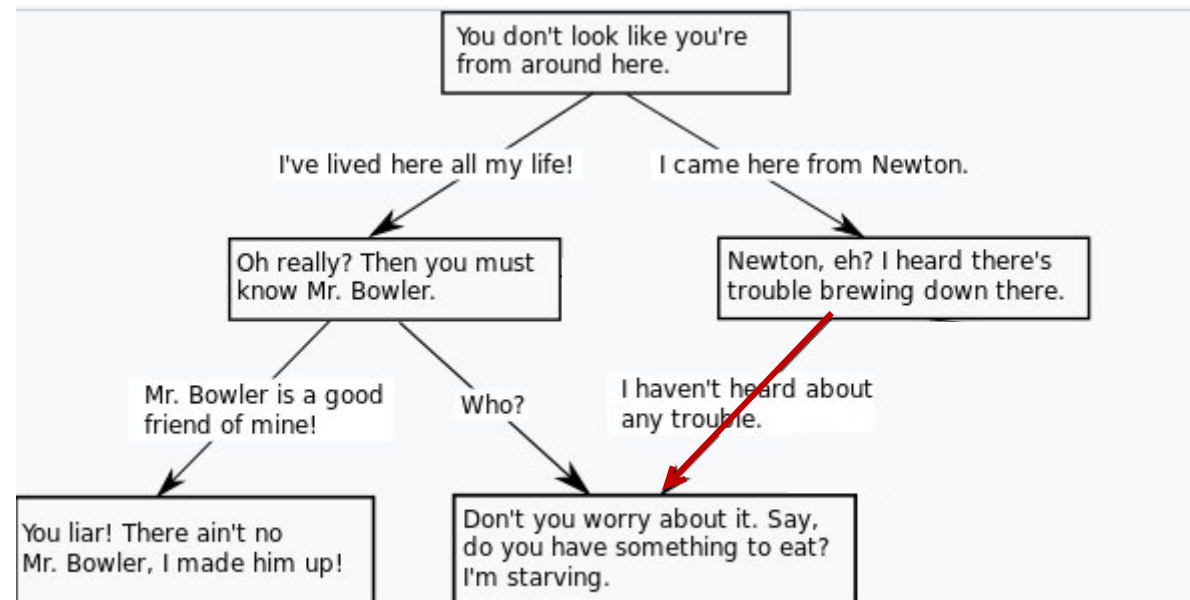
- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten sind durch die Verknüpfung zwischen den Knoten  $v \in V$  und  $w \in V$  erreichbar sind
- Ein Baum ist ein zusammenhängender Graph, bei dem eine Ebene über diesem ist
- Jeder Knoten hat genau einen Elternknoten, der eine Ebene über diesem ist
- Die Höhe  $h$  eines Baumes ergibt sich durch die höchste Ebene  $E$  eines Blattes mit  $h = E + 1$
- Die maximale Anzahl an Kanten eines Baumes ist  $\|E\|_{max} = n - 1$  für  $n = \|V\|$

Wieso ist der dialogue tree unten gar kein echter Baum?



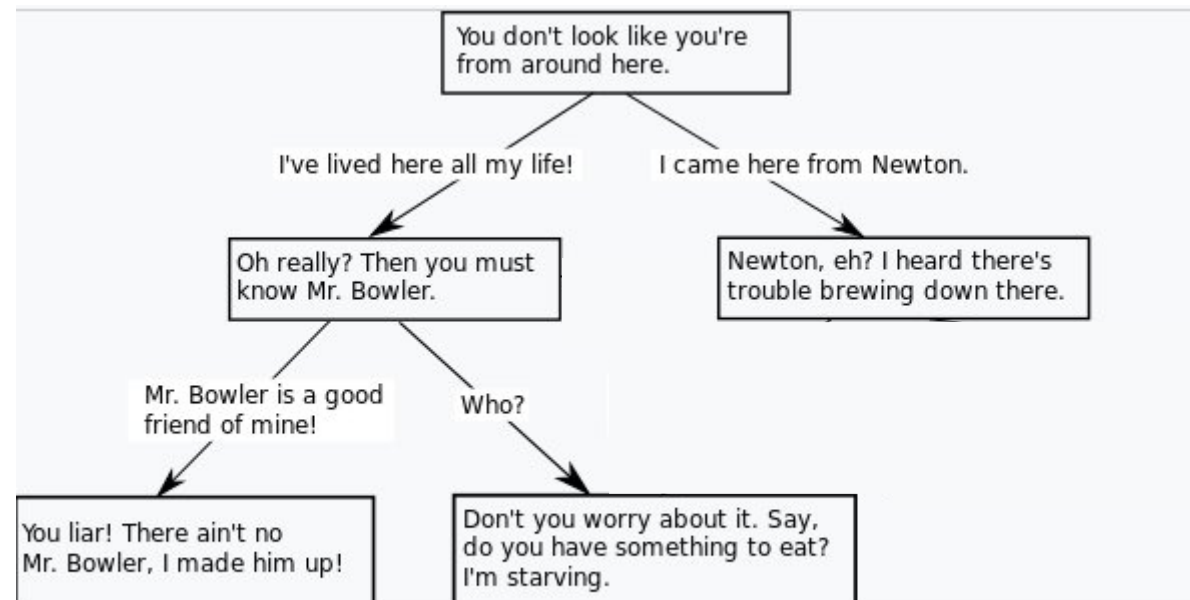
Gibt es zwischen der Wurzel und einem Knoten mehr als einen oder keinen Pfad, so handelt es sich um einen Graphen

- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine Wurzel (root)  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind
  - Jeder Knoten (Kind) (außer  $w$ ) hat genau einen Vorgänger (Vater), der eine Ebene über diesem ist
  - Die Höhe  $h$  eines Baumes ergibt sich durch die höchste Ebene  $E$  eines Blattes mit  $h = E + 1$
  - Die maximale Anzahl an Kanten eines Baumes ist  $\|E\|_{max} = n - 1$  für  $n = \|V\|$



An example of a simple dialogue tree

- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine Wurzel (root)  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind
  - Jeder Knoten (Kind) (außer  $w$ ) hat genau einen Vorgänger (Vater), der eine Ebene über diesem ist
  - Die Höhe  $h$  eines Baumes ergibt sich durch die höchste Ebene  $E$  eines Blattes mit  $h = E + 1$
  - Die maximale Anzahl an Kanten eines Baumes ist  $\|E\|_{max} = n - 1$  für  $n = \|V\|$



An example of a simple dialogue tree



- **Definition:** Ein Baum  $T$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, v_2, \dots, v_N\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die Knoten (nodes) repräsentieren die Elemente, die Kanten die Verknüpfung zwischen den Elementen. Ein Baum erfüllt folgenden Bedingungen:
  - Ein Baum hat genau eine Wurzel (root)  $w$  von welcher aus alle Knoten (nodes)  $V$  erreichbar sind
  - Jeder Knoten (Kind) (außer  $w$ ) hat genau einen Vorgänger (Vater), der eine Ebene über diesem ist
  - Die Höhe  $h$  eines Baumes ergibt sich durch die höchste Ebene  $E$  eines Blattes mit  $h = E + 1$
  - Die maximale Anzahl an Kanten eines Baumes ist  $\|E\|_{max} = n - 1$  für  $n = \|V\|$
- **Sonstiges**
  - Ein Pfad ist eine Folge von unterschiedlichen Knoten, die durch Kanten des Baums verbunden sind
  - Gibt es zwischen der Wurzel und einem Knoten mehr als einen oder keinen Pfad, so handelt es sich um einen **Graphen**
  - Falls jeder Knoten nur eine bestimmte Anzahl  $n$  an direkten Nachfolgern haben darf, die in einer bestimmten Reihenfolge vorliegen müssen, spricht man von einem  $n$ -ären Baum (Beispiel binärer Baum hat ein Knoten immer genau zwei Nachfolger)

# Baumstruktur - Grundbegriffe



## ■ Pfadlänge Baum ist 17

Ebene 1:  $1 + 1 + 1 + 1 + \dots$  Ebene 0

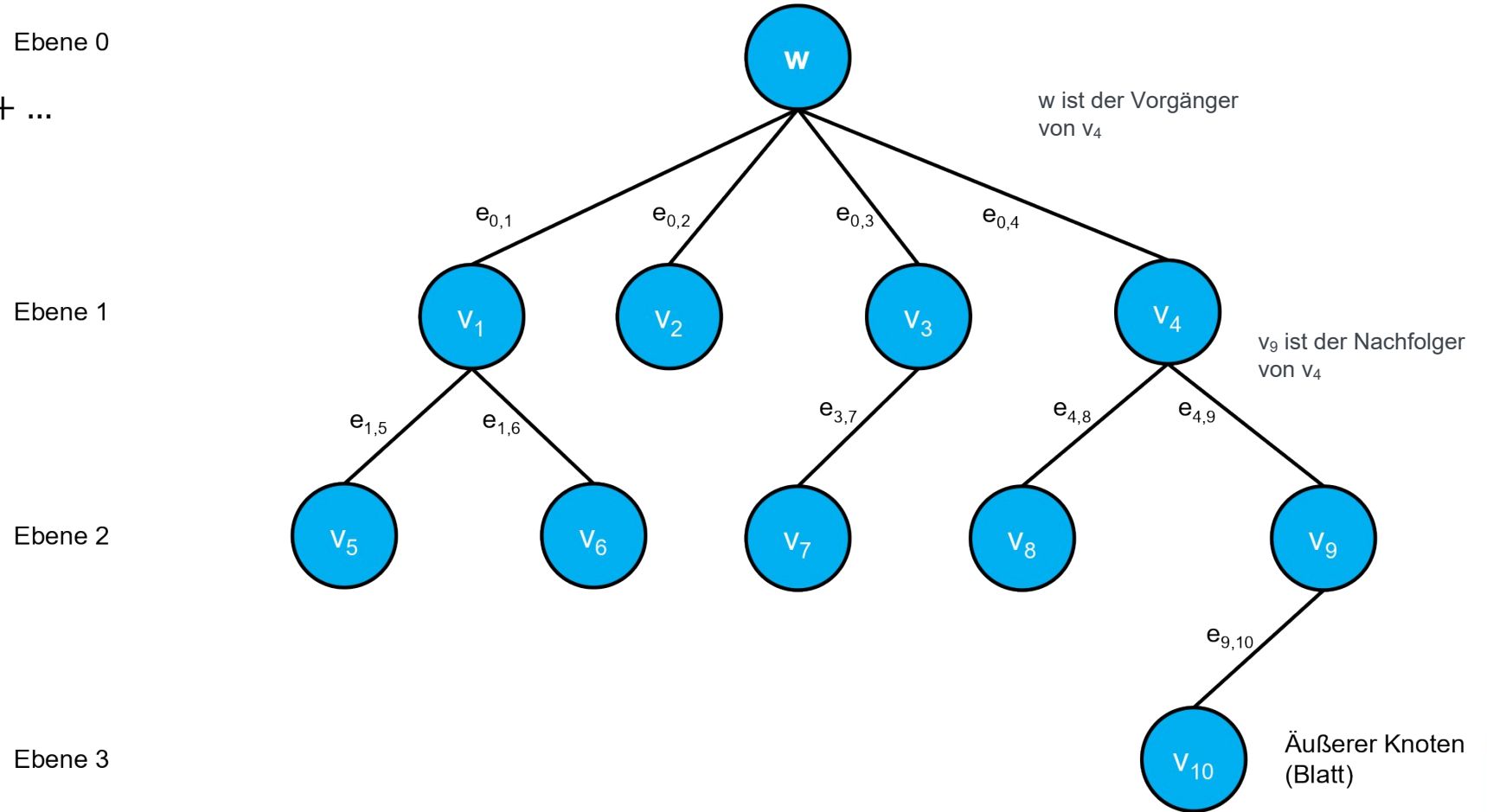
Ebene 2:  $2 + 2 + 2 + 2 + 2 + \dots$

Ebene 3: 3

= 17

## ■ Anzahl Ebenen ist 3

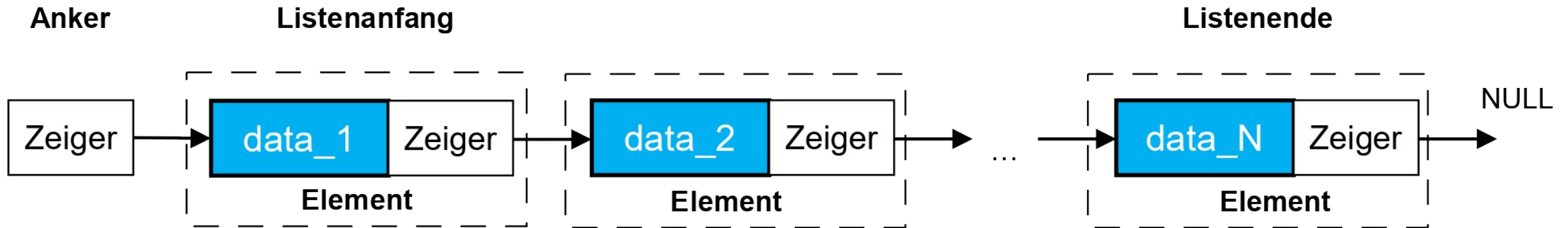
## ■ Baumhöhe ist 4



# Listenstruktur - Grundbegriffe



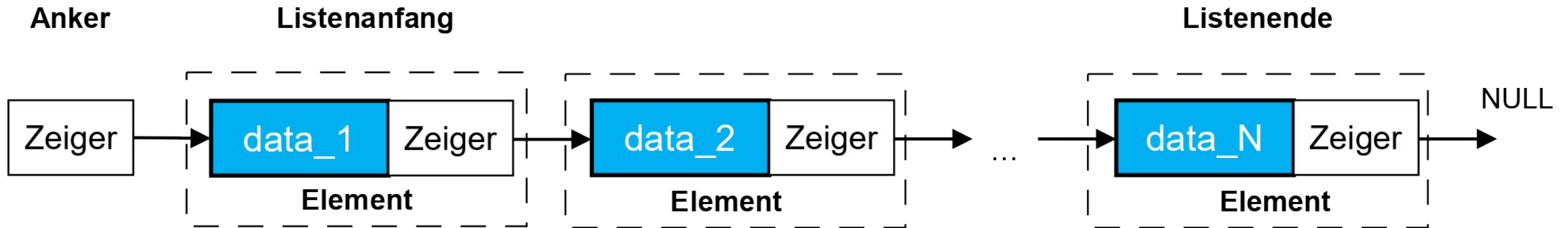
Eine **verkettete Liste** ist eine Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.



# Listenstruktur - Grundbegriffe



Eine **verkettete Liste** ist eine **dynamische** Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.





C++

- **Datentyp `struct`:** Dieser Datentyp erlaubt uns die Auflistung unterschiedlicher Datentypen zusammenzufassen. Somit können wir inhaltlich sinnvolle Zusammengehörigkeiten zwischen einzelnen Datentypen modellieren. Die Definition erfolgt außerhalb von Funktionen (`main`, ..) mit dem Schlüsselwort `struct`.

- **Beispiel:**

```
struct Student {  
    string first_name;    // Vorname  
    string last_name;     // Nachname  
    int matric_num;       // Matrikelnummer  
};  
  
void BasicUsage()  
{  
    Student new_student;    // Erstellen der Datenstruktur newStudent vom Typ Student  
    new_student.first_name = "Max";    // Zugriff über Punktoperator  
    new_student.last_name = "Mustermann";    // String-Konstanten mit double quotes ("  
    new_student.matric_num = 4711;    // Führende Nullen nicht möglich!  
}
```

- Das Beispiel verdeutlicht auch, dass die **Schreibweise von Bezeichnern** immer wichtiger wird. In diesem Beispiel wurde der "Google C++ Style Guide" [1] verwendet.





- **Übung 1:** Implementieren Sie ein C++ Programm, welches:
  - abfragt, wie viele Studierende mit Vorname, Nachname und Matrikelnummer erfasst werden sollen,
  - eine Erfassung von maximal 10 Studierenden vorsieht,
  - diese nacheinander erfasst und
  - strukturiert als Liste ausgibt.

# Struct – Lösung (1)



```
// Problemgröße handhaben
int num_student;
cout << "Wieviele Studierende sollen angelegt werden? >";
cin >> num_student;

if (num_student < 1)
{
    cout << "Okay, keine Eingabe." << endl;
    return;
}

if (num_student > 10)
{
    cout << "Anzahl Studierende wurde auf 10 begrenzt." << endl;
    num_student = 10;
}
```

# Struct – Lösung (2)



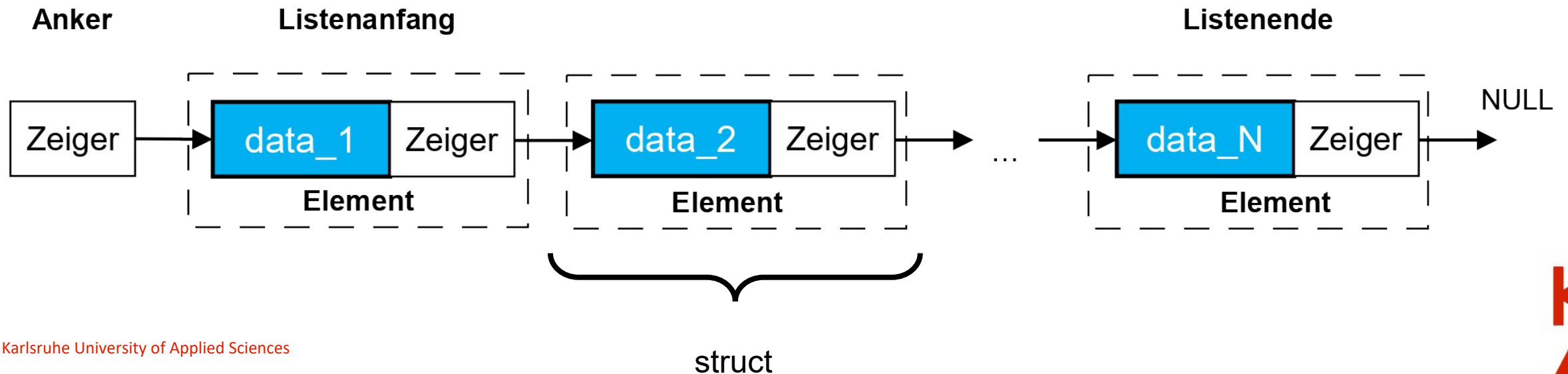
```
// Erfassung
Student student[10];
for (int i = 0; i < num_student; i++)
{
    // Erfassen
    Student temp;
    cout << endl << "Erfassung von Studierenden #" << i + 1 << endl;
    cout << "Vorname>"; cin >> temp.first_name;
    cout << "Nachname>"; cin >> temp.last_name;
    cout << "Matrikelnr.>"; cin >> temp.matric_num;

    // Abspeichern
    student[i] = temp;
}

// Ausgabe
cout << endl << "Ausgabe der Daten:" << endl;
for (int i = 0; i < num_student; i++)
{
    cout << "Studierender #" << i + 1 << ": "
        << student[i].first_name << ", "
        << student[i].last_name << ", "
        << student[i].matric_num << "."
        << endl;
}
```



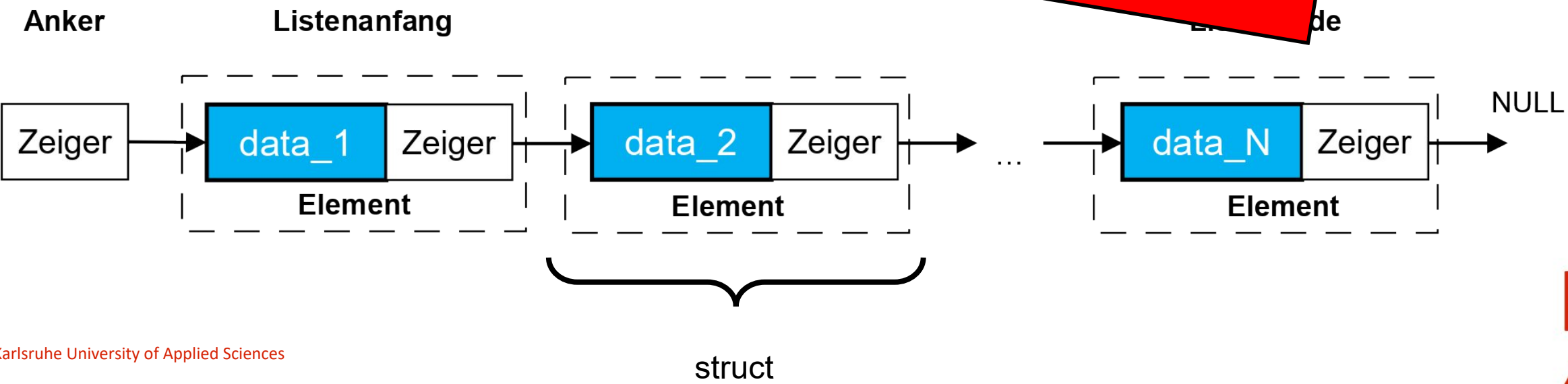
Eine **verkettete Liste** ist eine Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.



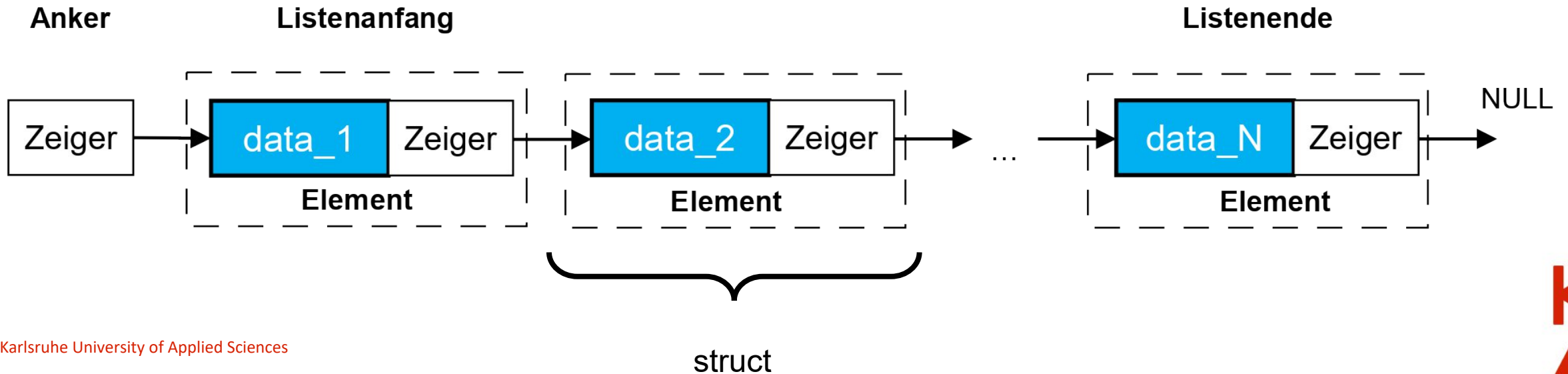


Eine **verkettete Liste** ist eine Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.

Wie müsste eine solche Struktur definiert werden?



```
struct ListenElement {  
    int data;  
    ListenElement* next;  
};
```

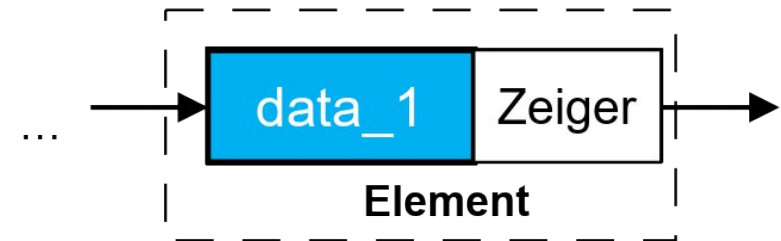


# Listenstruktur



```
ListenElement ErstesElement;
```

```
struct ListenElement {  
    int data;  
    ListenElement* next;  
};
```



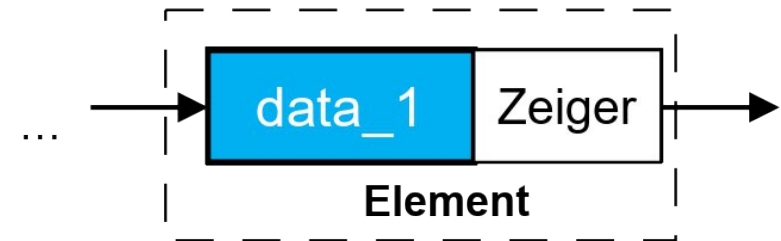
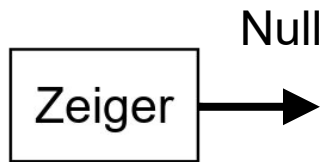


# Listenstruktur



```
ListenElement ErstesElement;  
  
ListenElement* root = nullptr;
```

```
struct ListenElement {  
    int data;  
    ListenElement* next;  
};
```

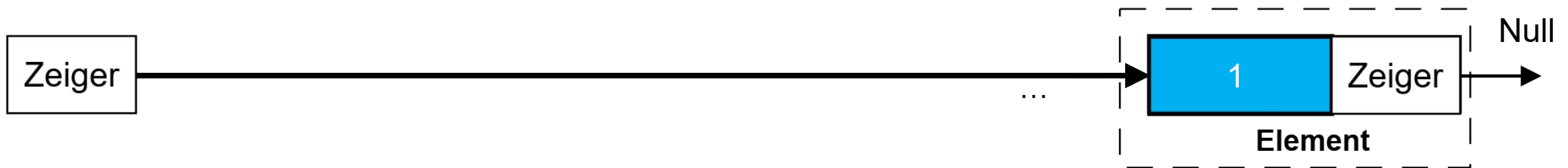


# Listenstruktur



```
ListenElement ErstesElement;  
  
ListenElement* root = nullptr;  
  
root = &ErstesElement;  
  
ErstesElement.data = 1;  
ErstesElement.next = nullptr;
```

```
struct ListenElement {  
    int data;  
    ListenElement* next;  
};
```

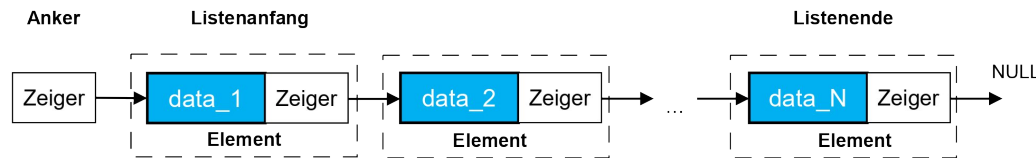


# Einfache und Doppelte Verkettung



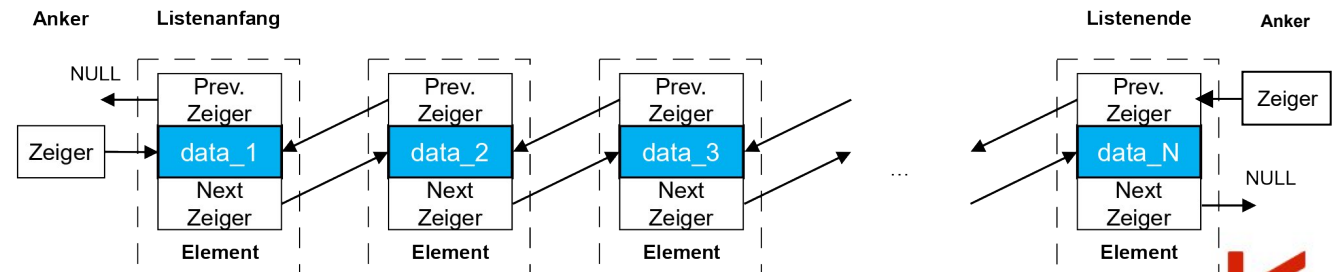
## Einfach verkettete Liste

- **Eigenschaften:** Datenelemente sind durch einen Zeiger (vorwärts oder rückwärts) verkettet.
- **Vorteile:**
  - Einfache Verwaltung, keine Kontroll-Struktur, nur ein Anker.
- **Nachteile:**
  - Operationen wie Delete() sind teurer.
  - Suchen teuer, nur in einer Richtung.



## Doppelt verkettete Liste

- **Eigenschaften:** Anstatt eines Zeigers auf Vorgänger- oder Nachfolgerelement, hält jedes Element einen Zeiger auf den Vorgänger als auch auf den Nachfolger
- **Vorteile:**
  - Kann von Listenanfang als auch Listenende durchsucht werden.
  - Schnelleres Löschen, da Vorgänger bekannt ist.
- **Nachteile:**
  - Pro Element müssen zwei Zeiger gespeichert werden; Ebenso müssen bei Operationen zwei Zeiger angepasst werden (Anker, Kontrollstruktur)

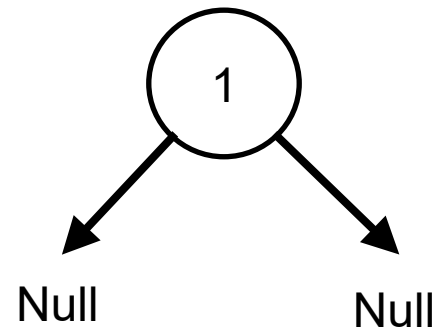


```
Knoten* Anker = nullptr;
```

```
Knoten child;  
child.data = 1;  
child.left = nullptr;  
child.right = nullptr;
```

Anker → Null

```
struct Knoten{  
    int data;  
    Knoten* left;  
    Knoten* right;  
};
```

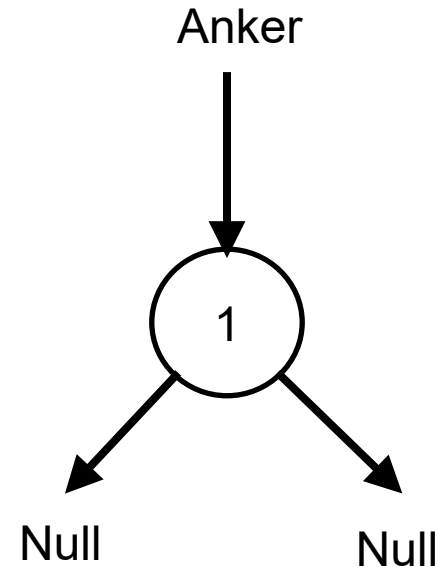


```
Knoten* Anker = nullptr;
```

```
Knoten child;  
child.data = 1;  
child.left = nullptr;  
child.right = nullptr;
```

```
Anker = &child;
```

```
struct Knoten{  
    int data;  
    Knoten* left;  
    Knoten* right;  
};
```



# Baumstruktur

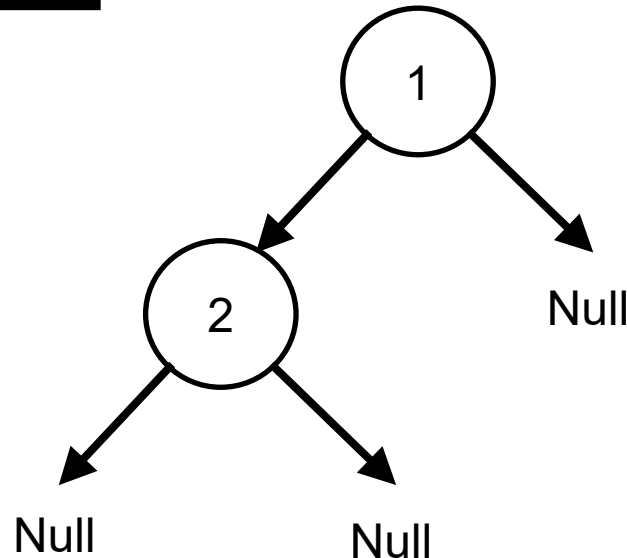


```
Knoten* Anker = nullptr;
```

```
Knoten E1, E2;  
E1.data = 1;  
E1.left = &E2;  
E1.right = nullptr;  
E2.data = 2;  
E2.left = nullptr;  
E2.right = nullptr;
```

Anker → Null

```
struct Knoten{  
    int data;  
    Knoten* left;  
    Knoten* right;  
};
```



# Baumstruktur

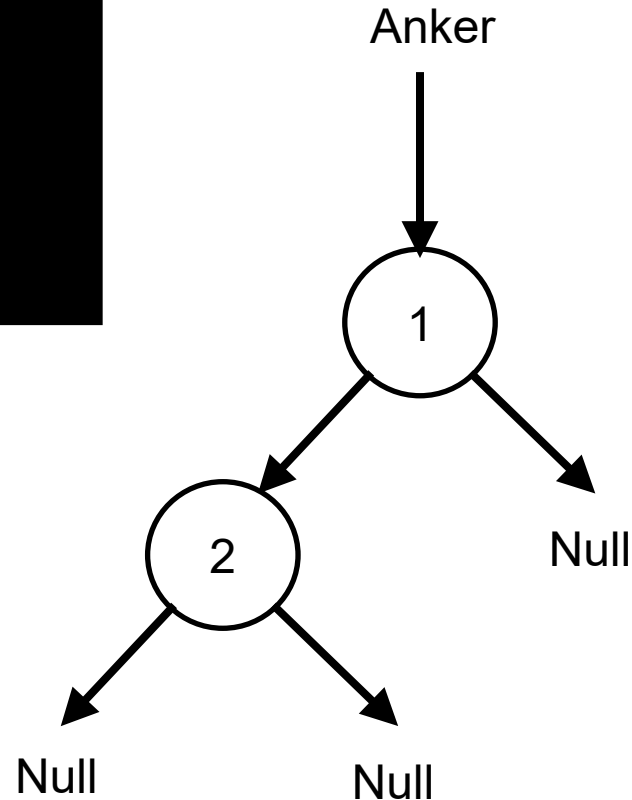


```
Knoten* Anker = nullptr;
```

```
Knoten E1, E2;  
E1.data = 1;  
E1.left = &E2;  
E1.right = nullptr;  
E2.data = 2;  
E2.left = nullptr;  
E2.right = nullptr;
```

```
Anker = &E1;
```

```
struct Knoten{  
    int data;  
    Knoten* left;  
    Knoten* right;  
};
```









```
#include <iostream>
using namespace std;
void ausgabe(int* ptr) { cout << *ptr << endl; }
void ausgabe2(int* ptr) { cout << *ptr << endl; *ptr = 8; }

int main() {
    // Wiederholung Info1
    int a = 10;
    int* ptr = &a;    /// ptr -> a
    cout << ptr << endl; // Inhalt der Variablen ptr
    cout << sizeof(ptr) << endl; // wie gross ist die Speicherstelle des Pointers
    cout << &a << endl; // Adresse von A
    cout << ptr << endl; // Inhalt der Variablen ptr
    cout << *ptr << endl; // Inhalt der Variablen auf die ptr zeigt
    cout << &ptr << endl; // Adresse der Variablen ptr

    // Ausgabe des Wertes "10", aber wie????
    // ausgabe( ??? );
    // Möglichkeit 1:
    ausgabe(&a);
    // Möglichkeit 2:
    ausgabe(ptr);    // Wieso ist hier a 8?? Klausuraufgabe?
    cout << a << endl;

    return 0; // Rückgabewert
}
```





```
#include <iostream>
using namespace std;
void ausgabe(int* ptr) { cout << *ptr << endl; }
void ausgabe2(int* ptr) { cout << *ptr << endl; *ptr = 8; }

int main() {
    int a = 10;
    int* ptr_a = &a;    /// ptr_a -> a
    int** ptr_ptr_a = &ptr_a; /// ptr_ptr_a -> ptr_a -> a
    int*** ptr_ptr_ptr_a = &ptr_ptr_a; /// ptr_ptr_ptr_a -> ptr_ptr_a -> ptr_a -> a
    cout << ptr_a << endl; // Inhalt der Variablen ptr_a
    cout << *ptr_ptr_a << endl; // Pointer auf Pointer dereferenziert....
    cout << **ptr_ptr_ptr_a << endl; // Pointer auf Pointer dereferenziert ....

    void (*ptr_f)(int*); // Pointer auf Funktion
    ptr_f = ausgabe;
    ptr_f(&a);

    return 0;
}
```



```
#include <iostream>
using namespace std;
int main() {
    // Strukturen einführen
    struct Ware {
        string name = "Kekse";
        float EK = 0.9f;
        double VK = 22.9;
    };

    Ware MeineErsteWare;
    cout << sizeof(MeineErsteWare) << endl;
    cout << &MeineErsteWare << endl;
    MeineErsteWare.name = "Schoki";
    MeineErsteWare.VK = 2.2;
    cout << MeineErsteWare.name << endl;

    Ware Eis, Cola, Bier; // 3 Waren erzeugen
    Ware Alles[3]; // Array mit 3 Waren erzeugen
    Alles[0].name = "nix";
    return 0; // Rückgabewert
}
```





```
#include <iostream>
using namespace std;
int main() {
    // verkettete Liste
    struct Element {
        int Daten;
        Element* next;
    };
    Element A;
    A.Daten = 1;
    Element B;
    B.Daten = 2;
    Element C;
    C.Daten = 3;
    Element* Anker = &A;
    A.next = &B;
    B.next = &C;
    C.next = nullptr;

    Element* tmp = Anker;
    while (tmp != nullptr) {
        cout << tmp->Daten << ", ";
        // hier fehlt noch Code... -> Übungsaufgabe
    }
    return 0; }
```



# reinterpret\_cast-Operator



## Explizite Typenumwandlung:

- Benutzen des **static\_cast**-Operators

```
char c {'x'};  
int i = static_cast<int>(c);           // Typumwandlung char → int
```

bedeutet, dass der Wert der Variablen `i` eine `int`-Repräsentation der `char`-Variablen `c` ist. Andere, einfachere Schreibweisen sind ebenfalls möglich:

```
i = int(c);                           // oder
```

```
i = (int) c;
```

**überprüft** Typenverträglichkeit!

- Benutzen des **reinterpret\_cast**-Operators

**Verzichtet** weitgehend **auf Prüfung** der Typenverträglichkeit!

Ermöglicht, dass ein beliebiger Zeiger in einen anderen Zeigertyp konvertiert wird.

Ermöglicht es außerdem, einen beliebigen ganzzahligen Typ in einen beliebigen Zeigertyp und umgekehrt zu konvertieren.

```
#include <iostream>  
using namespace std;  
  
int main(){  
    int a = 10;  
    int* ptr_int_a = &a;  
    double* ptr_double_a = nullptr;  
  
    ptr_double_a = reinterpret_cast<double*>(ptr_int_a);  
    cout << *ptr_int_a << endl;  
  
    return 0;  
}
```

# Foreach / range-based for loop



The **foreach** loop in C++ or more specifically, **range-based for loop** was introduced with the **C++11**. This type of for loop structure eases the traversal over an iterable data set.

```
int arr[]={1,2,3,4,5};    //array initialization
cout<<"The elements are: ";
for(int i : arr) {
    cout << i << " ";
}
```